



UNIVERSIDAD DE MÁLAGA



Graduado en Ingeniería Informática

Desarrollo de una aplicación web para el apoyo a la elección de asignaturas optativas basado en técnicas de análisis de datos y el machine learning.

Development of a web application to support the selection of optional subjects based on data analysis techniques and machine learning.

Realizado por
Jesús Chincoa González

Tutorizado por
Rafael Marcos Luque Baena
Enrique Soler Castillo

Departamento
Lenguajes y Ciencias de la Computación

MÁLAGA, diciembre de 2025



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADUADO EN INGENIERÍA INFORMÁTICA

**Desarrollo de una aplicación web para el apoyo a la
elección de asignaturas optativas basado en técnicas de
análisis de datos y el machine learning**

**Development of a web application to support the
selection of optional subjects based on data analysis
techniques and machine learning**

Realizado por

Jesús Chíncoa González

Tutorizado por

Rafael Marcos Luque Baena

Enrique Soler Castillo

Departamento

Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA
MÁLAGA, DICIEMBRE DE 2025

Fecha defensa: diciembre de 2025

Abstract

This Bachelor's Thesis presents the development of a web application designed to assist students in choosing elective courses by leveraging data analysis and machine learning techniques. The platform, built with Angular on the frontend and Python for machine learning logic, allows students to input their academic record number and upload a PDF file containing their transcript, as available on the Student Portal, to receive personalized course recommendations based on historical grade comparisons with previous cohorts, as well as a statistical estimation of the probability of securing a spot in each course. The solution implements a data pipeline encompassing cleaning, normalization, and supervised learning algorithms, evaluated through precision metrics. The backend offers a RESTful API that manages Oracle data persistence and the inference service, while the frontend provides an intuitive user interface. Experimental results show over 85 % recommendation accuracy and an average response time of under 200 ms. This system not only efficiently guides students but also enables the academic institution to identify demand patterns for elective courses.

Keywords: web application, machine learning, recommendation, Angular, Python.

Resumen

El presente Trabajo de Fin de Grado describe el desarrollo de una aplicación web orientada a guiar al alumnado en la selección de asignaturas optativas mediante técnicas de análisis de datos y aprendizaje automático. La plataforma, construida con Angular para el frontend y Python para la lógica de Machine Learning, permite al estudiante introducir su número de expediente y un fichero PDF con el expediente académico disponible en el Portal de Estudiantes y obtener recomendaciones personalizadas basadas en la comparación de sus calificaciones históricas con las de cohortes previas, así como una estimación estadística de probabilidad de plaza en cada asignatura. Para ello, se emplea un pipeline de datos que incluye limpieza, normalización y modelado con algoritmos supervisados, evaluados mediante medidas de precisión. El backend expone una API REST que gestiona la persistencia en Oracle y el servicio de inferencia, mientras que el frontend ofrece una interfaz intuitiva. Los resultados demuestran una precisión superior al 85 % en la recomendación de asignaturas, y un tiempo de respuesta medio inferior a 200 ms. Este sistema no solo orienta de manera eficiente a los estudiantes, sino que también facilita al centro académico la identificación de patrones de demanda.

Palabras clave: aplicación web, aprendizaje automático, recomendación, Angular, Python.

Índice

1. Introducción	9
1.1. Motivación	9
1.2. Descripción del problema	9
1.3. Objetivos	10
1.3.1. Objetivo general	10
1.3.2. Objetivos específicos	10
1.4. Alcance y limitaciones	11
1.5. Metodología	11
1.6. Estructura de la memoria	11
2. Estado del arte y tecnologías	13
2.1. Revisión de soluciones existentes	13
2.2. Tecnologías valoradas	14
2.3. Justificación de la elección tecnológica	15
3. Análisis y diseño	17
3.1. Requisitos	17
3.1.1. Requisitos funcionales	17
3.1.2. Requisitos no funcionales	18
3.2. Casos de uso y actores	18
3.3. Diagrama de secuencia	26
3.4. Arquitectura de componentes	27
3.5. Diagrama de clases	28
4. Implementación	31
4.1. Base de datos	31
4.1.1. Vistas	31
4.1.2. Tablas	33
4.2. Backend y API	34
4.2.1. Estructura del proyecto en Python	34
4.2.2. Conexión a la base de datos	35
4.2.3. Lectura y procesamiento del PDF del expediente	35
4.2.4. Probabilidad de Acceso	36
4.3. Lógica de Machine Learning	36

4.3.1.	Preprocesado de datos	36
4.3.2.	Modelo de recomendación	37
4.3.3.	Evaluación de rendimiento	39
4.4.	Frontend en Angular	40
4.4.1.	Estructura de módulos y componentes	40
4.4.2.	Páginas de la aplicación y capturas de interfaz	40
4.5.	Contenerización e integración con Docker	43
4.6.	Protección de datos	44
5.	Pruebas y validación	47
5.1.	Estrategia de pruebas	47
5.2.	Herramientas	47
5.3.	Plan de pruebas	47
5.3.1.	Pruebas de uso	47
5.3.2.	Pruebas unitarias	50
5.4.	Resultados y métricas	51
6.	Resultados y discusión	53
6.1.	Análisis de métricas de recomendación	53
6.2.	Evaluación de usabilidad (pruebas manuales)	54
6.3.	Rendimiento del sistema	56
7.	Conclusiones y Líneas Futuras	59
7.1.	Conclusiones	59
7.2.	Líneas futuras	59
Apéndice A.	Manual de Instalación	63
A.1.	Requisitos del Sistema	63
A.2.	Guía de Instalación con Docker	63
A.2.1.	Clonar el repositorio	63
A.2.2.	Instalar Docker Desktop y Docker Compose	63
A.2.3.	Construir los contenedores y levantar	64
A.2.4.	Bajar los contenedores (sin borrar volúmenes)	64
A.2.5.	Limpiar al completo (elimina volúmenes e imágenes locales)	64
A.3.	Guía de Instalación en equipo local	64
A.3.1.	Clonar el repositorio	64
A.3.2.	Instalar Python y pip	65
A.3.3.	Configuración del entorno Python	65

A.3.4.	Instalar dependencias del Backend e iniciar	65
A.3.5.	Instalar dependencias del Frontend y desplegar	66
A.3.6.	Acceder a la aplicación	66
Apéndice B. Manual de Usuario		67
B.1.	Para cualquier usuario	67
B.2.	Como alumno	68
B.3.	Como administrador	71

1

Introducción

1.1. Motivación

El creciente número de estudiantes matriculados en la Escuela Técnica Superior de Ingeniería Informática de la Universidad de Málaga ha provocado una elevada demanda de plazas en las asignaturas optativas. La gestión manual de matriculación por parte de la Secretaría resulta cada vez más compleja y los estudiantes carecen de información precisa sobre el contenido real de las optativas, lo que puede generar desajustes entre expectativas y resultados académicos.

Además, observando la información que hace pública la Universidad de Málaga en su página web [1], actualmente la tasa de ocupación de las titulaciones es superior al 100 %, es decir, ingresan más alumnos que plazas se ofertan. En el último curso, hablamos de un 107 % para Ingeniería de la Salud, un 109 % para Ingeniería del Software y un 111 % para Ingeniería Informática.

Esto se añade a la limitación por plazas que tienen las optativas de la escuela, y resulta en un complejo proceso de matriculación con Secretaría, ya que las asignaturas más demandadas se solicitan cada vez más al haber más alumnos cada año, por lo que también se realizan más modificaciones de matrícula cada año.

1.2. Descripción del problema

En la Escuela Técnica Superior de Ingeniería Informática existe una limitación administrativa habitual: cada asignatura optativa dispone de un número máximo de plazas (en el contexto de este trabajo se considera un límite de **35 plazas** por optativa). Esta restricción provoca que determinadas asignaturas de gran atractivo queden rápidamente colapsadas, mientras que otras con menos visibilidad permanezcan infrautilizadas.

Además, el alumnado generalmente dispone únicamente de la información contenida en la guía docente, la cual en algunos casos sí contiene información sobre asignaturas que se recomienda haber superado previamente, pero en otros casos no se añaden esas indicaciones. En muchos casos, toma decisiones con escasa información sobre la planificación real, el enfoque docente o la aplicabilidad práctica de la materia en su itinerario formativo.

Estas circunstancias generan dos problemas principales: por una parte, el estudiante afronta una elección compleja y con incertidumbre sobre si la asignatura cumplirá sus expectativas; por otra, el centro docente carece de información precisa sobre la demanda potencial y los patrones de elección del alumnado que le permitan anticipar la ocupación de plazas y planificar recursos docentes. La literatura sobre sistemas de recomendación para asignaturas señala que tanto la falta de información como la heterogeneidad de perfiles dificultan la personalización y la predicción de demanda, lo que refuerza la necesidad de soluciones que combinen análisis de datos y métodos predictivos [2]-[4].

El objetivo de la aplicación propuesta es, por tanto, doble: (i) ofrecer al estudiante una referencia orientativa sobre las asignaturas optativas que más se adecúan a su perfil académico —basada en el historial de calificaciones y en patrones detectados en cohortes previas—; y (ii) proporcionar al centro académico indicadores agregados sobre las asignaturas más consultadas y las probabilidades estimadas de obtención de plaza, de modo que pueda anticipar la demanda y mejorar la planificación. Esta doble perspectiva busca reducir la asimetría informativa en la elección de optativas y aportar herramientas de apoyo tanto a la toma de decisión individual como a la gestión del centro.

1.3. Objetivos

1.3.1. Objetivo general

Desarrollar una solución web que servirá para realizar un estudio inicial sobre la viabilidad de aplicar aprendizaje automático y técnicas de clustering para encontrar patrones comunes entre una muestra cerrada de estudiantes previamente matriculados en el Centro, y que permita clasificar al alumno y poder realizar predicciones de afinidad y, al realizarse en el contexto de un TFG, servirá para afianzar los conocimientos del desarrollo en Angular para el frontend y Python para el motor de Machine Learning.

1.3.2. Objetivos específicos

- Diseñar el modelo de datos para almacenar calificaciones y solicitudes de matrícula.
- Implementar un pipeline de preprocesado de datos.
- Desarrollar modelos supervisados que clasifiquen alumnos y generen recomendaciones.
- Exponer los servicios de recomendación y predicción mediante una API REST en Python.
- Garantizar la privacidad y seguridad de los datos del alumno siguiendo lo estipulado en la Ley Orgánica de Protección de Datos.

- Crear una interfaz de usuario intuitiva en Angular.
- Crear un panel de administrador con acceso a las consultas realizadas en la aplicación.

1.4. Alcance y limitaciones

El sistema se limitará a estudiantes de los diferentes grados de la Escuela Técnica Superior de Ingeniería Informática en la Universidad de Málaga. Se utilizarán exclusivamente datos históricos internos y no se considerarán integraciones con plataformas externas de gestión académica. Además, en esta fase no se incluye recomendación basada en el contenido temático de las asignaturas. Las predicciones y el clustering se realizarán exclusivamente con la información de la vista proporcionada por los tutores de este Trabajo de Fin de Grado.

1.5. Metodología

Dada la naturaleza iterativa y ágil del proyecto, se adopta la metodología **SCRUM**. Este marco de trabajo favorece la entrega incremental de funcionalidades y la adaptación continua a los cambios de requisitos, definiendo una serie de ciclos iterativos o sprints. Se ha descompuesto el proyecto en una serie de subtarear que se abordarán en cada uno de los ciclos.

Dentro de cada sprint, el enfoque elegido ha sido, por un lado, el desarrollo completo de un conjunto de subtarear para el proyecto, y por otro, integrar el trabajo nuevo con el realizado previamente, y corregir distintos problemas que hayan podido surgir durante este proceso. Para poder llevar esta dinámica de trabajo, la duración del sprint se ha fijado en cuatro semanas.

En las iteraciones finales, se ha probado el funcionamiento de la aplicación y el control de alumnos que acceden a los datos subidos, protegiendo los expedientes académicos subidos con la verificación por parte del usuario del número de expediente y el DNI.

Para la gestión del código y el control de versiones, se ha utilizado Git y GitHub, lo que ha permitido el trabajo desde distintos dispositivos, así como un control robusto de los cambios realizados en la aplicación.

1.6. Estructura de la memoria

La memoria se articula en los siguientes capítulos:

1. **Introducción.** Motivación, objetivos, alcance y organización de la memoria.
2. **Estado del arte y tecnologías.** Análisis de soluciones existentes y justificación del stack tecnológico.
3. **Análisis y diseño.** Requisitos, diagramas UML (casos de uso, clases y componentes) y modelo de datos.

4. **Implementación.** Explicación formal del funcionamiento de la aplicación, incluyendo parte frontal, API, algoritmos de Machine Learning y almacenamiento de datos.
5. **Pruebas y validación.** Estrategias de testing y resultados obtenidos.
6. **Resultados y discusión.** Evaluación de métricas de recomendación y experiencia de usuario.
7. **Conclusiones y líneas futuras.** Posibles ideas para seguir mejorando la aplicación entregada.
8. **Bibliografía.**
9. **Apéndices.** Manual de instalación y Manual de usuario.

2

Estado del arte y tecnologías

2.1. Revisión de soluciones existentes

En la literatura y en el mercado académico se han propuesto diversas herramientas para apoyar la elección de asignaturas optativas. Entre las más destacadas se encuentran:

- **CoursePicker:** plataforma que emplea técnicas de filtrado colaborativo para sugerir asignaturas basadas en los intereses explícitos del usuario [4], [5]. No obstante, este enfoque no considera el rendimiento académico previo, lo que limita la personalización basada en el historial del estudiante.

The screenshot shows the Course Picker web application. At the top, there is a navigation bar with the Course Picker logo, a login link, and links to help, credits, feedback, official catalog, and OCW. Below the navigation bar, a message states: "Course Picker needs your help to make this a better tool for you. Please take this quick survey! Thank you!".

The main content area features a search bar with the text "Search for a class:". Below the search bar, it displays "2244 Classes" and sorting options: "ordenados por: undefined luego por..." and a button to "agrupar según orden".

On the left side, there is a sidebar with a list of classes under the heading "browse classes". The list includes: 5 6 7, 12 14, 18 20, 21H, 22, CMS, ES, MAS, SCM, WGS, Hass-S, and Physics-2.

On the right side, there is a "Full Calendar View" section with a calendar grid. Below the calendar, there are three buttons: "Export classes to Google Calendar", "Fill in my recitations!", and "Export to CourseRoad".

The main content area displays two course listings. The first listing is for "1.00 - Engineering Computation and Data Science" (Spring semester). It includes a description: "Presents engineering problems in a computational setting with emphasis on data science and problem abstraction. Covers exploratory data analysis and visualization, filtering, regression. Building basic machine learning models (classifiers, decision trees, clustering) for smart city applications. Labs and programming projects focused on analytics problems faced by cities, infrastructure, and environment. Students taking graduate version complete additional assignments and project work." It also lists the instructor "J. Williams", prerequisites "18.01 or 18.014 or 18.01A", level "Undergraduate", units "3-2-7", and course evaluation "for 1.00". Below the course description, there are two sections: "Lecture Sections" with a button to "Add" and "Lab Sections" with a button to "Add".

The second listing is for "1.001 - Engineering Computation and Data Science" (Spring semester). It includes a description: "Presents engineering problems in a computational setting with emphasis on data science and problem abstraction. Covers exploratory data analysis and visualization, filtering, regression. Building basic machine learning models (classifiers, decision trees, clustering) for smart city applications. Labs and programming projects focused on analytics problems faced by cities, infrastructure and environment. Students taking graduate version will complete additional assignments and project work." It also lists the instructor "J. Williams" and prerequisites "18.01 or 18.014 or 18.01A".

Figura 1: Course Picker

- **ElectiveAdvisor:** aplicación que aplica algoritmos de clustering para agrupar perfiles

de estudiantes y sugerir asignaturas relacionadas [6], [7]. Requiere grandes volúmenes de datos para obtener buenos resultados y su precisión no supera el 70 %, según datos disponibles en publicaciones académicas.

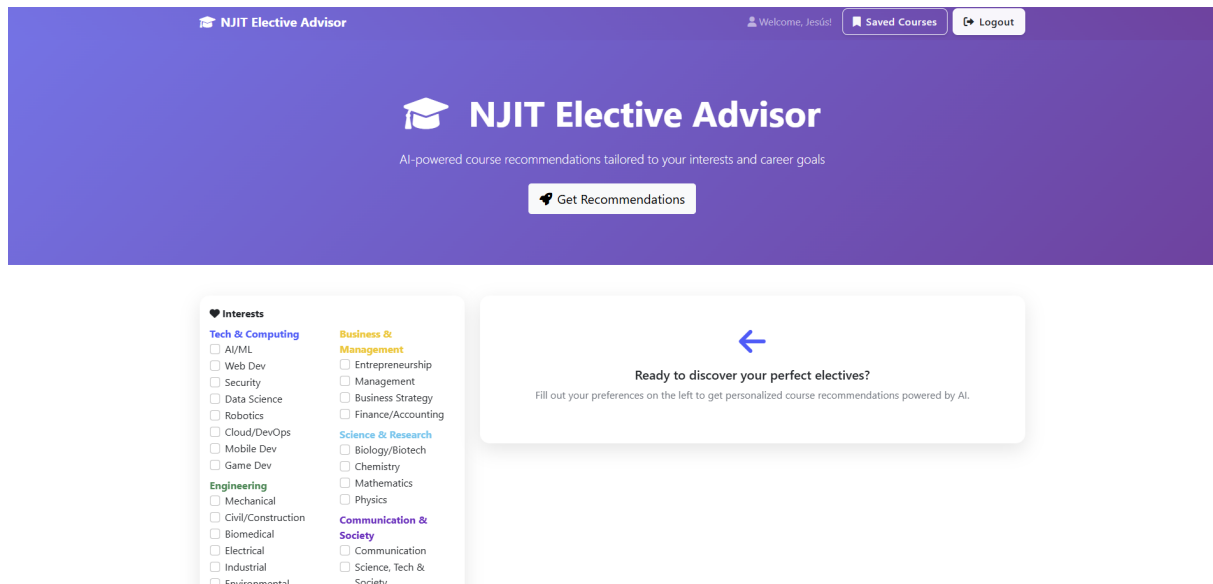


Figura 2: Elective Advisor

A diferencia de estas propuestas, la solución desarrollada en este TFG se enfoca específicamente en combinar el rendimiento académico histórico del estudiante con técnicas de aprendizaje supervisado para generar recomendaciones más ajustadas a su perfil. Además, se proporciona una estimación de la probabilidad de acceso a cada asignatura basada en datos reales de matrícula, lo que ofrece un valor añadido tanto para el estudiante como para la institución. Aunque se trata de una primera versión, la integración de ambas dimensiones —recomendación y predicción de plaza— diferencia esta propuesta de las soluciones revisadas.

2.2. Tecnologías valoradas

A continuación se resumen las tecnologías analizadas para cada capa del sistema:

■ Frontend:

- **Angular:** framework con inyección de dependencias, buenas prácticas MVC y comunidad activa.
- **React:** librería ligera, alto rendimiento pero requiere más configuración de arquitectura.

- **Vue.js:** curva de aprendizaje suave, pero menor soporte empresarial.
- **Backend:**
 - **Spring Boot (Java):** maduro y robusto, pero aumenta la complejidad de despliegue.
 - **Node.js + Express:** servidor ligero en JavaScript, buena integración con frontend, pero menos madurez en ML.
 - **Python + FastAPI (o Flask):** sintaxis sencilla y potente ecosistema de ML (scikit-learn, pandas).
- **Acceso a datos:**
 - **Vista SQL sobre base de datos institucional:** se emplea una vista ya definida por la Secretaría Académica, a la que se accede desde herramientas como Oracle SQL Developer.
 - **SQL Developer:** cliente gráfico que permite ejecutar consultas sobre bases de datos Oracle, facilitando la exploración, exportación y análisis previo de datos.
- **Machine Learning:**
 - **scikit-learn:** modelos supervisados clásicos, documentación extensa.
 - **TensorFlow/Keras:** apto para redes neuronales, pero excesivo para regresión y clasificación simples.

2.3. Justificación de la elección tecnológica

El desarrollo de esta aplicación requiere una arquitectura robusta, modular y escalable, con tecnologías ampliamente adoptadas en el entorno académico y profesional. En este contexto, se han seleccionado las siguientes soluciones, que se detallan y justifican a continuación:

Angular [8] ha sido elegido como framework para el desarrollo del frontend debido a su estructura basada en componentes reutilizables, el soporte nativo para TypeScript y su potente sistema de routing y servicios. Estas características facilitan el mantenimiento a largo plazo y permiten escalar la aplicación conforme crezcan sus funcionalidades. Además, cuenta con herramientas integradas como Angular CLI y una comunidad activa que garantiza acceso a actualizaciones, buenas prácticas y documentación extensa.

Para el desarrollo del backend, se opta por **Python** en combinación con el framework **FastAPI** [9], por su sintaxis limpia, bajo umbral de entrada y compatibilidad con librerías especializadas en machine learning como *scikit-learn* y *pandas* [6], [10]-[12]. FastAPI permite construir APIs RESTful de forma eficiente y segura, genera documentación automática mediante

OpenAPI (Swagger) y proporciona alto rendimiento gracias al uso de ASGI (Asynchronous Server Interface Gateway), que permite el manejo eficiente de múltiples conexiones concurrentes y la ejecución de tareas asíncronas mediante las palabras clave `async/await` de Python, ofreciendo así un mayor rendimiento y escalabilidad frente a frameworks tradicionales como Flask o Django.

Acceso a datos: con el propósito de mantener los datos almacenados conjuntamente, no se han considerado otras tecnologías y se han implementado las distintas **tablas de datos** usando una base de datos **Oracle** [13]. En las tablas, se almacena información relacionada con los expedientes que los usuarios envían a la aplicación, así como de las distintas consultas que realizan en ésta, para poder ser consultadas por los administradores.

En cuanto al motor de aprendizaje automático, se ha optado por la biblioteca **scikit-learn**, que proporciona una amplia gama de algoritmos supervisados y no supervisados, entre ellos *KMeans*, *k-NN* o *regresión logística*. Estas técnicas permiten analizar el historial académico del estudiante y generar recomendaciones fundamentadas en datos reales. Además, scikit-learn facilita la integración con otras bibliotecas y herramientas de análisis como NumPy y Matplotlib, lo que ha permitido evaluar el rendimiento de los modelos mediante métricas como precisión, recall y F1-score.

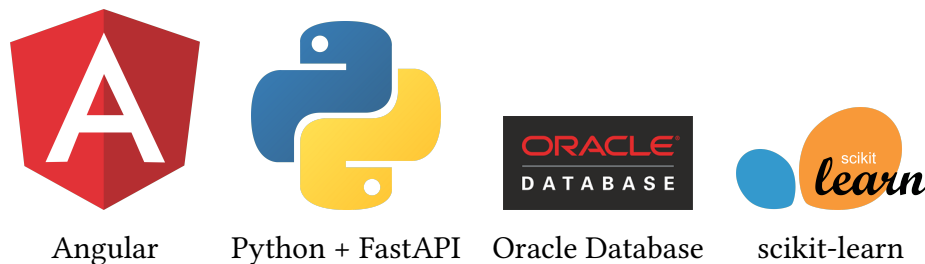


Figura 3: Tecnologías seleccionadas para el desarrollo del proyecto.

3

Análisis y diseño

3.1. Requisitos

En esta sección se definirán los requisitos funcionales y no funcionales identificados en el proyecto, así como los distintos casos de uso derivados de estos.

3.1.1. Requisitos funcionales

1. RF1: El sistema deberá permitir al usuario subir un fichero PDF con su expediente académico.
2. RF2: El sistema extraerá y normalizará automáticamente las asignaturas y notas para su posterior análisis.
3. RF3: El sistema ofrecerá al estudiante un listado de asignaturas optativas de su titulación de las que poder elegir para consultar.
4. RF4: El sistema generará recomendaciones de asignaturas basadas en modelos de Machine Learning.
5. RF5: El sistema mostrará al usuario la probabilidad estimada de obtener plaza en cada asignatura.
6. RF6: El sistema almacenará de forma segura la información sobre el número de expediente académico y el DNI para identificar al alumno, así como los datos del expediente subido.
7. RF7: El sistema permitirá al usuario actualizar su expediente, así como eliminarlo de la base de datos.
8. RF8: El sistema comprobará las credenciales del administrador, y le redirigirá al panel de administrador si existe el usuario como administrador.

3.1.2. Requisitos no funcionales

- RNF1: El tiempo de respuesta de la API no deberá superar los 300 ms en el 95 % de las peticiones.
- RNF2: La interfaz deberá ser intuitiva para el usuario, y dirigir de forma clara a cada uno de los casos de uso.
- RNF3: Se debe cumplir en todo momento y para toda la información tratada la LOPD.

3.2. Casos de uso y actores

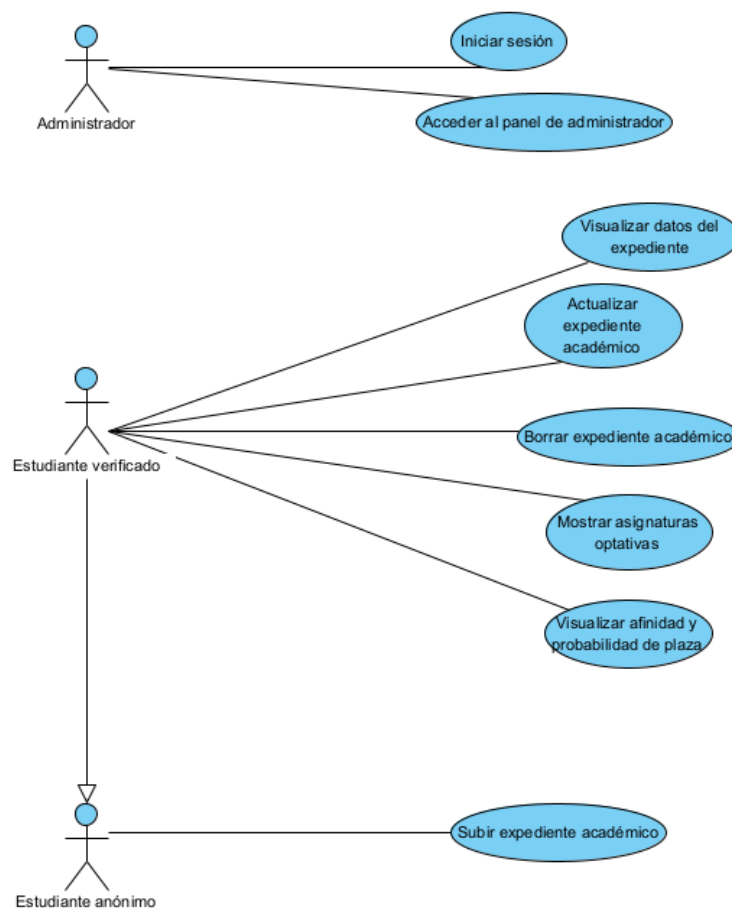


Figura 4: Diagrama de casos de uso principales.

Como se observa en la figura 4, los casos de uso que se han indicado en la memoria se representan con los casos realizados con los actores Estudiante anónimo, Estudiante verificado y Administrador. Un Estudiante verificado es aquel del que se tiene almacenado su expediente académico, por lo que se puede cambiar de Estudiante a Estudiante verificado al subir el

expediente, y de Estudiante verificado a Estudiante cuando se solicite el borrado de dicho expediente de la base de datos. Un Administrador es una persona con acceso a una cuenta que está registrada como administrador en la base de datos, y que podrá consultar los datos del panel de administrador.

La verificación de administrador se realiza mediante una función llamada VALIDA_USUARIO otorgada por uno de los tutores en la cuenta Oracle. Para comprobar si un usuario es administrador, el backend llama a esta función con las credenciales de Usuario y Contraseña, y la función verifica si las credenciales son correctas y si el usuario tiene permisos de administrador. Si ambas son correctas, devuelve 1, y si falla alguna o ambas devuelve 0.

Cuadro 1: Caso de uso: Subir expediente académico

Caso de uso	Subir expediente académico
Resumen	Un estudiante sube por primera vez su expediente.
Actores	Estudiante anónimo.
Precondición	El estudiante no debe tener su expediente guardado en la base de datos.
Postcondición	El estudiante queda guardado con su número de expediente y su DNI y tiene acceso a sus datos y puede realizar consultas.
Curso Normal	1. Estudiante accede a la aplicación 2. Estudiante indica su número de expediente y 4 últimos dígitos del DNI + letra. 3. Estudiante sube su expediente académico 4. El sistema guarda las credenciales del usuario y los datos del expediente y redirige al estudiante
Curso Alternativo	1. El número de expediente y/o 4 últimos dígitos del DNI + letra añadidos por el estudiante no coinciden con los del expediente, por lo que se le notifica del error y no se registra
Observaciones	Los datos del DNI se almacenan hasheados en la base de datos.

Cuadro 2: Caso de uso: Visualizar datos del expediente

Caso de uso	Visualizar datos del expediente
Resumen	La aplicación muestra al estudiante las calificaciones reflejadas en el PDF.
Actores	Estudiante verificado.
Precondición	El estudiante debe tener su expediente guardado en la base de datos.
Postcondición	El sistema muestra todas las calificaciones del estudiante (solo la última obtenida en cada asignatura).
Curso Normal	1. El estudiante accede a su perfil. 2. La aplicación le solicita que verifique su número de expediente y 4 últimos dígitos del DNI + letra. 3. La aplicación redirige al estudiante a su perfil, y le muestra sus calificaciones
Curso Alternativo	1. El número académico y/o 4 últimos dígitos del DNI + letra no coinciden con los almacenados, por lo que se notifica el error y no se redirige. 2. El estudiante ya se ha verificado previamente en la sesión y sigue identificado, por lo que se le redirige directamente.
Observaciones	

Cuadro 3: Caso de uso: Actualizar expediente académico

Caso de uso	Actualizar expediente académico
Resumen	El estudiante vuelve a subir su expediente con nuevas asignaturas.
Actores	Estudiante verificado.
Precondición	El estudiante debe tener su expediente guardado en la base de datos.
Postcondición	Las nuevas calificaciones se muestran en la aplicación
Curso Normal	▪ El estudiante pulsa el botón Actualizar expediente. ▪ La aplicación le redirige a la página para subir el expediente. ▪ El estudiante sube el expediente actualizado. ▪ La aplicación le redirige a su página de perfil.
Curso Alternativo	▪ El estudiante abandona la página sin subir nada, por lo que al volver a acceder, siguen los datos anteriores del estudiante.
Observaciones	No se realiza un borrado del estudiante de la base de datos, se hace un <i>merge</i> de los datos obtenidos para almacenar los cambios.

Cuadro 4: Caso de uso: Borrar expediente académico

Caso de uso	Borrar expediente académico
Resumen	El estudiante borra sus datos guardados en la base de datos
Actores	Estudiante verificado.
Precondición	El estudiante debe tener su expediente guardado en la base de datos
Postcondición	Se borra el expediente de la base de datos y se borra de la lista de alumnos verificados el número de expediente del alumno y 4 últimos dígitos del DNI + letra.
Curso Normal	▪ El estudiante pulsa el botón Eliminar expediente. ▪ La aplicación envía una notificación de verificación al estudiante. ▪ El estudiante confirma su deseo de borrar sus datos de la aplicación. ▪ La aplicación borra los datos del estudiante, manda un mensaje de confirmación y le redirige a la landing page.
Curso Alternativo	▪ El usuario decide no borrar sus datos, por lo que se mantiene en su página con sus datos.
Observaciones	No se borran los datos sobre las consultas realizadas.

Cuadro 5: Caso de uso: Mostrar asignaturas optativas

Caso de uso	Mostrar asignaturas optativas
Resumen	La aplicación muestra al estudiante las asignaturas optativas que tiene guardadas de su titulación.
Actores	Estudiante verificado.
Precondición	El estudiante debe tener su expediente guardado en la base de datos.
Postcondición	La aplicación muestra al estudiante las asignaturas optativas que tiene guardadas de su titulación para que el estudiante pueda realizar las consultas
Curso Normal	▪ El estudiante entra en la página con los datos de su expediente. ▪ La aplicación le muestra, junto a sus calificaciones, un listado de asignaturas optativas de su titulación para que pueda realizar peticiones.
Curso Alternativo	▪ Si no se han podido cargar asignaturas optativas de la titulación del estudiante, se le muestra un mensaje de error.
Observaciones	

Cuadro 6: Caso de uso: Visualizar afinidad y probabilidad de plaza

Caso de uso	Visualizar afinidad y probabilidad de plaza
Resumen	El estudiante selecciona una asignatura de las que le aparecen, y la aplicación calcula la probabilidad de acceso y la afinidad con ésta.
Actores	Estudiante verificado.
Precondición	El estudiante debe tener su expediente guardado en la base de datos.
Postcondición	La aplicación muestra la afinidad y la probabilidad de plaza en una tarjeta
Curso Normal	▪ El estudiante pulsa una asignatura de las que se le muestra ▪ La aplicación calcula, para dicha asignatura, la probabilidad de acceso del estudiante y la afinidad con dicha asignatura. ▪ La aplicación le muestra los resultados al estudiante.
Curso Alternativo	▪ Si la asignatura es muy reciente para tener datos históricos, se le notifica del fallo al usuario.
Observaciones	Para las consultas de administrador, si un estudiante consulta varias veces la misma asignatura, solo se guarda una vez.

Cuadro 7: Caso de uso: Iniciar sesión

Caso de uso	Iniciar sesión
Resumen	El administrador inicia sesión en la aplicación para poder acceder al panel de administrador
Actores	Administrador.
Precondición	El usuario debe no haber iniciado sesión previamente ni como estudiante ni como administrador.
Postcondición	El administrador inicia sesión y se le da acceso al panel de administrador.
Curso Normal	▪ El usuario pulsa en el boton Login admin ▪ La aplicación le redirige a la página de login ▪ El usuario indica sus credenciales. ▪ La aplicación consulta si el usuario está registrado como administrador en la base de datos. ▪ La aplicación redirige al usuario al panel de administrador.
Curso Alternativo	▪ Si no se encuentra al usuario en la lista de administradores, la aplicación muestra un error al usuario.
Observaciones	La validación se realiza mediante la función VALIDA_USUARIO, implementada en el usuario de SQL Developer.

Cuadro 8: Caso de uso: Acceder al panel de administrador

Caso de uso	Acceder al panel de administrador
Resumen	El administrador accede al panel con las consultas exclusivas para él.
Actores	Administrador.
Precondición	El administrador debe haber iniciado sesión en la aplicación.
Postcondición	El administrador accede a los datos del panel.
Curso Normal	▪ El administrador pulsa el botón Panel admin. ▪ La aplicación redirige al administrador al panel.
Curso Alternativo	▪ Si un Estudiante anónimo o Estudiante verificado intenta acceder mediante URL al panel de administrador, se le redirige automáticamente a la landing page.
Observaciones	

3.3. Diagrama de secuencia

Para facilitar la comprensión del caso de uso referente a la subida del expediente académico, en este diagrama de secuencia se muestra cómo el Estudiante envía el PDF y se pueden dar dos casos: que el PDF no sea válido y le deje en la página actual para que suba el correcto, o que lea las notas del expediente y le redirija hacia su página, ahora como Estudiante verificado.

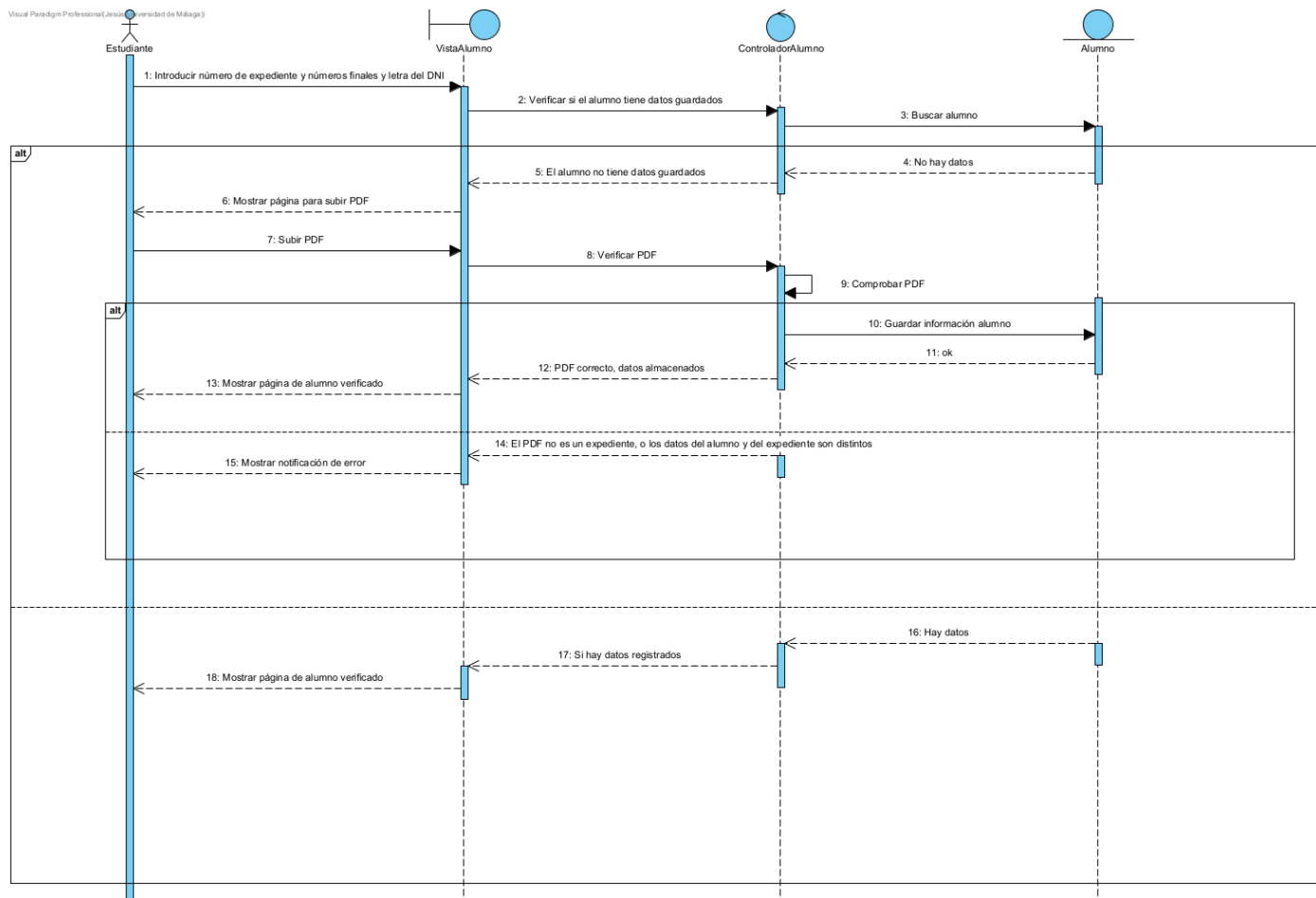


Figura 5: Diagrama de secuencia del caso de uso subir expediente académico.

3.4. Arquitectura de componentes

En la Figura 6 se representa la comunicación que se realiza internamente en la aplicación. Dentro de ésta, podemos detallar aspectos de cada una:

- **Frontend:** es la parte visual de la aplicación, y con la que interactuarán los usuarios. Contiene todas las páginas que componen la aplicación, y los servicios que se comunican con la API REST del backend para obtener los datos y mostrarlos al alumno.
- **Base de datos:** perteneciente a **afrodita**, el servidor Oracle de la UMA, contiene las vistas proporcionadas con datos para poder realizar el proceso de Machine Learning, y los datos que se almacenan sobre la aplicación.
- **Backend (Machine Learning):** es la parte del backend que se encarga de realizar todas las funciones y todos los procesos de limpieza de datos, para poder realizar los cálculos de probabilidad de entrada y afinidad para los alumnos.

- **Backend (API REST):** es la parte del backend que comunica todos los demás componentes: obtiene los datos necesarios de la base de datos, recibe y resuelve peticiones de información por parte del frontend y el algoritmo de Machine Learning y, en general, controla el acceso a la información de la aplicación.

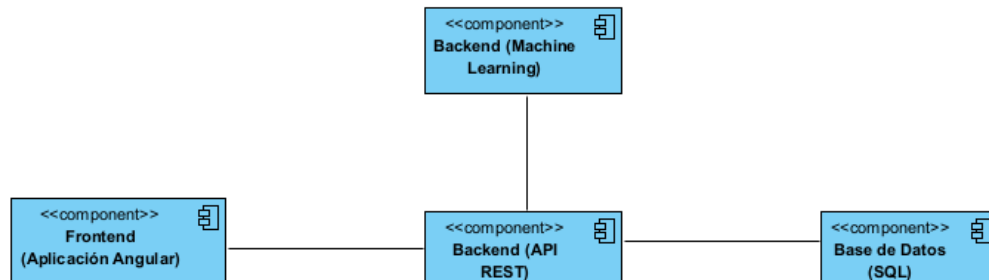


Figura 6: Diagrama de componentes de la aplicación (Frontend, Backend, ML).

3.5. Diagrama de clases

En la Figura 7 se detallan todos los elementos con los que trabaja la aplicación como clases que contienen ciertos atributos que representan propiedades de éstas, o funciones que realizan para la aplicación, así como la cardinalidad de éstos:

- **Asignatura:** cada una de las asignaturas de las que se tiene registro en la base de datos. De ellas se conocen su **código**, su **nombre** y los **créditos** que cuesta. Cada recomendación es de una asignatura, y cada calificación le corresponde a una asignatura concreta.
- **Calificación:** la nota obtenida por un alumno en una asignatura. Se conoce la **asignatura** a la que corresponde, la **nota** obtenida y la **convocatoria** en la que se obtuvo dicha calificación. Todas las calificaciones de un alumno se almacenan en su expediente.
- **Expediente:** colección de calificaciones de un alumno matriculado. Se extrae de éste el **número del expediente** y las **calificaciones** que contiene. Cada estudiante tiene un expediente
- **Estudiante:** es el usuario normal de la aplicación, representa a un alumno cualquiera. Posee el **número de expediente**, que le relaciona con este, y su **DNI**, que le permite ser identificado por la aplicación. Un estudiante puede realizar tantas peticiones de recomendación como quiera.

- **Recomendación:** es el par (probabilidad de entrada, afinidad) de un estudiante para una asignatura. El estudiante puede solicitar varias recomendaciones al recomendador, y que sean
- **Recomendador:** corresponde a la parte frontal de la aplicación. En ésta, un estudiante solicita que se le generen recomendaciones, lo que llama a **obtenerRecomendacion()**. El recomendador puede mostrar varias recomendaciones al mismo tiempo, y crea una conexión con el API Controller
- **API Controller:** API REST implementada en el backend que representa el punto donde se reunen todos los datos de la aplicación para que el recomendador pueda solicitarlos. Ésta crea una conexión, además, con el gestor de datos y el modelo de Machine Learning.
- **Modelo ML:** representa el algoritmo de aprendizaje supervisado que se crea para poder calcular la afinidad de un alumno con una asignatura. Dicho algoritmo se puede volver a entrenar en caso de cambiar la entrada de datos que se le proporciona, y es el que predice si una asignatura es afín o no con un estudiante, así como su probabilidad de entrada en una asignatura.
- **Gestor de Datos:** representa la base de datos de la aplicación. En ella, se tienen todos los datos de los expedientes que han sido añadidos a la base de datos, así como de las consultas que los distintos estudiantes han realizado en la aplicación.

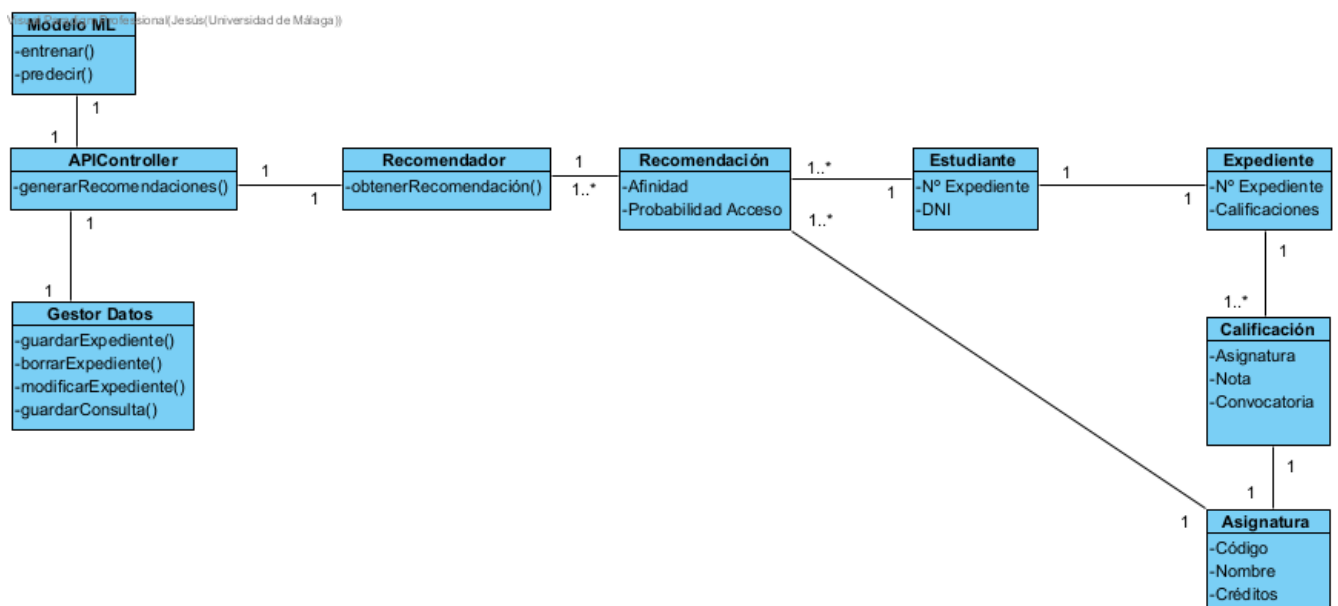


Figura 7: Diagrama de clases UML del módulo de dominio.

Los diagramas anteriores definen la estructura lógica y la interacción entre los distintos módulos de la aplicación. El modelo de datos se basa en una vista SQL institucional que agrupa las entidades **Estudiante**, **Asignatura**, **Calificación**, permitiendo realizar consultas eficientes para el entrenamiento y la inferencia de los modelos de Machine Learning.

4

Implementación

Como se ha mencionado anteriormente, el desarrollo de la aplicación se ha centrado en tres partes principales: la base de datos, el backend (la lógica de machine learning y la API REST) y el frontend.

4.1. Base de datos

Los tutores proporcionaron un usuario de base de datos Oracle, al que se accede mediante SQL Developer, con la información que a continuación se menciona.

4.1.1. Vistas

Se han otorgado las siguientes vistas para realizar la implementación tanto del backend como del frontend:

- **V_CALIFICACIONES:** Almacena los datos históricos de los alumnos que se usan para alimentar al cluster y realizar toda la lógica de machine learning para la afinidad, así como la probabilidad de acceso a la asignatura. Número de tuplas en el momento de la implementación: **96914**. En total hay **2953 alumnos** distintos en la vista, por lo que de media cada alumno tiene unos **33 registros** en la vista. Los alumnos de la vista se reparten en:
 - 1084 del Grado en Ingeniería Informática. Plan 2010.
 - 681 del Grado en Ingeniería del Software. Plan 2010.
 - 524 del Grado en Ingeniería de la Salud.
 - 410 del Grado en Ingeniería de Computadores.
 - 99 del Máster Universitario en Ingeniería del Software e Inteligencia Artificial.
 - 85 del Máster Universitario en Ingeniería Informática.
 - 67 del Doble Grado en Ingeniería Informática y Matemáticas.
 - 3 del Máster Universitario en Transformación Digital de Empresas.

Columnas:

- CENTRO: VARCHAR2.
 - TITULACIÓN: VARCHAR2.
 - CURSOACADÉMICO: VARCHAR2.
 - CODIGOALUM: VARCHAR2.
 - CÓDIGOASIGNATURAUMA: VARCHAR2.
 - NOMBREASIGNATURA: VARCHAR2.
 - SEMESTRE: VARCHAR2.
 - CALIFICACIÓN: VARCHAR2.
 - CREDITOS: NUMBER.
 - NUM_CALIFICACIÓN: VARCHAR2.
 - CONVOCATORIAS: VARCHAR2.
- V_OPTATIVAS: Contiene distintas asignaturas optativas que se ofertan en el centro junto con el número de la titulación, así como si todavía se imparten o no. Esta información se usa para elegir las asignaturas que se le mostrarán al alumno en el frontend. Número de tuplas en el momento de la implementación: 67. De éstas:
- 12 corresponden al Grado en Ingeniería Informática plan 2023.
 - 23 corresponden al Grado en Ingeniería Informática plan 2010.
 - 4 corresponden al Grado en Ciberseguridad e Inteligencia Artificial.
 - 12 corresponden al Grado en Ingeniería del Software plan 2023.
 - 14 corresponden al Grado en Ingeniería de la Salud.
 - 1 corresponden al Grado en Ingeniería del Software plan 2010.
 - 1 corresponden al Grado en Ingeniería de Computadores.

Columnas:

- CURSO_ACADEMICO: VARCHAR2, no nulo.
- TITULACION: NUMBER, no nulo
- CODIGO: NUMBER, no nulo.
- REFERENCIA: VARCHAR2, no nulo.
- CREDITOS: NUMBER, no nulo.

- OFERTADA: CHAR, no nulo.
- NOMBRE: VARCHAR2, no nulo.
- **V_TITULACION:** Relaciona los códigos de las titulaciones con el nombre de la propia titulación y la cantidad de créditos totales de dicha titulación. Número de tuplas en el momento de la implementación: **9**. Las titulaciones que se almacenan actualmente en esta vista son:
 - Grado en Ingeniería Informática. Plan 2010.
 - Grado en Ingeniería del Software. Plan 2010.
 - Grado en Ingeniería de Computadores.
 - Grado en Ingeniería de la Salud.
 - Doble Grado en Ingeniería Informática y Matemáticas.
 - Doble Grado en Ingeniería Informática y Matemáticas. Plan 2023.
 - Grado en Ciberseguridad e Inteligencia Artificial.
 - Grado en Ingeniería del Software. Plan 2023.
 - Grado en Ingeniería Informática. Plan 2023.

Columnas:

- CODIGO: NUMBER; no nulo.
- NOMBRE: VARCHAR2, no nulo.
- CREDITOS: NUMBER, no nulo.

4.1.2. Tablas

Se han creado tres tablas para almacenar la información que obtiene la aplicación:

- **ADMIN INFO:** Guarda los datos de las consultas que se realizan, para mostrarlos posteriormente en el panel del administrador. Columnas:
 - ID_ALUMNO(el número de expediente): VARCHAR2, no nulo.
 - NOMBRESIGNATURA: VARCHAR2, no nulo.
 - TITULACION: VARCHAR2, no nulo.
 - AFINIDAD: NUMBER.
 - PROBABILIDAD: NUMBER.

- **ALUMNOS VERIFICADOS:** Almacena las parejas número de expediente con los 4 dígitos finales del DNI + letra, hasheados estos últimos por seguridad de la información. Si el alumno borra el expediente de la base de datos, también se eliminan sus datos de esta tabla. Columnas:
 - CODIGOALUM: VARCHAR2, no nulo.
 - DNI_HASH: VARCHAR2, no nulo.
- **INFORMES ALUMNOS:** Contiene los datos de los expedientes subidos por el alumno. Cada una de las filas contiene los datos que se obtienen del expediente: número de expediente del alumno, asignatura, créditos de la asignatura, curso académico en el que se cursó, convocatoria en la que se obtiene la nota, calificación obtenida en número y en texto (Suspendo, Aprobado, Notable, Sobresaliente, Matrícula de Honor, No Presentado). Columnas:
 - CODIGOALUM: VARCHAR2, no nulo.
 - NOMBREALSIGNATURA: VARCHAR2, no nulo.
 - CREDITOS: NUMBER.
 - CURSO_ACADEMICO: VARCHAR2, no nulo.
 - CONVOCATORIA: VARCHAR2, no nulo.
 - NUM_CALIFICACIÓN: VARCHAR2
 - CALIFICACIÓN: VARCHAR2

4.2. Backend y API

4.2.1. Estructura del proyecto en Python

El servidor se organiza siguiendo convenciones de FastAPI, con módulos para rutas, modelos y servicios:

- **app:** Módulo con todo el contenido de la aplicación, que permite exportarla al completo para el despliegue en Docker y los tests unitarios con pytest.
 - `main.py`: punto de entrada y configuración de la aplicación.
 - `routers/`: define los endpoints REST.
 - `models/`: esquemas de Pydantic para la validación de datos.
 - `funciones.py`: incluye los métodos de acceso a la base de datos, análisis y procesamiento de datos, machine learning y predicciones para acceder a ellos desde los diferentes endpoints y reducir el código de éstos.

- **test:** Contiene todos los tests unitarios realizados a los endpoints de la aplicación y a las funciones.

4.2.2. Conexión a la base de datos

Se utiliza oracledb y un pool de conexiones que se envía a los distintos ficheros de routers

El lifespan de la aplicación permite que, cuando se inicia el backend, se genere la conexión a la base de datos y se cree el cluster, lo que permite reducir la latencia de las peticiones.

4.2.3. Lectura y procesamiento del PDF del expediente

El expediente académico se obtiene del portal académico de la UMA, al cual se puede acceder desde la aplicación mediante un enlace.

Es importante ya que el formato del PDF se tiene en cuenta en el proceso de lectura, por lo que algún cambio en el nombre del PDF al descargarse o en la forma de redacción de éste puede generar incompatibilidad con la aplicación.

Para realizar este proceso, se utiliza la librería **pdfplumber** de Python, la cual permite extraer la información de las tablas y almacenarla en una lista, además de la información en la cabecera del documento, donde aparecen el número de DNI y el de expediente, los cuales se usan para comprobar que coinciden con los introducidos en el frontend.

En la materia: Sistemas Inteligentes II					
Denominación asignatura	Créditos	Curso Acad.	Convocatoria	Calif.Num.	Calif.Literal
Sistemas Inteligentes II	6	2022/23	MAY/2023	6.00	Aprobado
En el módulo: Formación básica					
En la materia: Empresa					
Denominación asignatura	Créditos	Curso Acad.	Convocatoria	Calif.Num.	Calif.Literal
Organización Empresarial	6	2020/21	JUN/2021	6.20	Aprobado
En la materia: Estadística					
Denominación asignatura	Créditos	Curso Acad.	Convocatoria	Calif.Num.	Calif.Literal
Métodos Estadísticos para la Computación	6	2020/21	JUN/2021	2.30	Suspense
	6	2020/21	SEP/2021		No presentado
	6	2023/24	JUN/2024	9.00	Sobresaliente
En la materia: Física					
Denominación asignatura	Créditos	Curso Acad.	Convocatoria	Calif.Num.	Calif.Literal
Fundamentos Físicos de la Informática	6	2020/21	FEB/2021	2.80	Suspense
	6	2020/21	SEP/2021		No presentado
	6	2023/24	ENE/2024	3.20	Suspense
	6	2023/24	FEB/2024	5.50	Aprobado
Fundamentos de Electrónica	6	2020/21	FEB/2021	2.90	Suspense
	6	2020/21	SEP/2021	2.30	Suspense
	6	2023/24	ENE/2024	5.80	Aprobado

Figura 8: Ejemplo de las tablas del fichero de expediente académico que contienen las calificaciones

La aplicación comprueba el nombre del PDF y, tras leer el contenido previo a las notas del expediente, las almacena todas y realiza un proceso de limpieza para almacenar solo una nota

de cada asignatura y que el nombre de asignatura sea correcto en todas. Tras finalizar este proceso, almacena toda la información en la tabla `INFORMES_ALUMNO`.

4.2.4. Probabilidad de Acceso

Esta funcionalidad del backend es la que más tarda en ejecutarse, debido al coste de buscar entre todos los alumnos de la vista matriculados en la asignatura consultada, su nota media y clasificarla por años. Comprobando tiempos de ejecución en consola, varía entre los 5 y los 15 segundos, siendo las más tardías las asignaturas con mayor número de registros en la vista. La probabilidad de entrada es igual al porcentaje de años en los que la nota media ponderada del alumno es superior a la nota de corte de dicho año, que se calcula como la nota media ponderada mínima de los alumnos matriculados cada año hasta el año anterior al de matricularse en dicha asignatura.

```
{'2019-20': 1.9, '2020-21': 1.65, '2021-22': 2.33, '2022-23': 2.12}
```

Figura 9: Resultado de la petición de notas de corte para la asignatura Visión por Computador

En la Figura 9, vemos que se obtiene, para cada año, una nota media. Ésto se ha implementado devolviendo en la petición que busca los alumnos matriculados en la asignatura también el año en el que se matricularon, y calculando la media ponderada de dichos alumnos el curso anterior a entrar en la asignatura.

La media ponderada se calcula, siguiendo el modelo de la UMA, como la suma del producto de cada una de las notas por el número de créditos que le corresponden, todo dividido por el total de créditos de la titulación.

Para evitar datos incompletos o anómalos, se descartan los alumnos que en su registro de `V_CALIFICACIONES` tengan pocas asignaturas aprobadas.

4.3. Lógica de Machine Learning

4.3.1. Preprocesado de datos

Se crea una matriz de alumnos con las distintas notas de la vista `V_CALIFICACIONES` en cada una de las asignaturas.

- En las columnas, las distintas asignaturas que están en la vista.
- En las filas, los códigos de alumnos

- El valor correspondiente a una fila y columna concretos es la nota del alumno en dicha asignaturas, o 0 en caso de no haberla cursado.

Esta matriz es sobre la que se realiza la búsqueda de los clusters, buscando grupos de alumnos con calificaciones similares para colocar el centro de cada cluster. Con los datos actuales de la vista, se crea una matriz de **2783 filas x 164 columnas**. Cada fila corresponde a un alumno de la vista V_CALIFICACIONES, y cada columna es una asignatura de la que se tienen datos en V_CALIFICACIONES.

4.3.2. Modelo de recomendación

Para elegir el modelo a usar para las recomendaciones, se ha realizado un estudio comparativo entre KMeans y Gaussian Mixture Model (GMM).

KMeans divide el conjunto de datos en K grupos o clústeres distintos, donde los puntos dentro de un clúster son los más similares entre sí. Cada clúster tiene un centroide, y cada punto pertenece al clúster cuyo centroide es más próximo a él, en este caso, el que tiene alumnos con notas más parecidas.

GMM asume que los distintos datos son generados mediante una combinación de distintas distribuciones normales de distintos parámetros. En lugar de trabajar con clústeres fijos como KMeans, GMM otorga a cada punto una probabilidad de pertenecer a cada uno de los distintos clústeres.

Con el apoyo de ChatGPT [14], se han creado dos scripts para elegir entre ambas opciones, comparando tres métricas utilizadas frecuentemente en la literatura: Silhouette Score (separación entre clústers), Davies–Bouldin Index (ratio de separación intra-clúster e inter-clúster) y Calinski–Harabasz Index (aumenta con la separación entre clústeres). Para ello, se le proporcionó mediante prompt las funciones para calcular las mencionadas métricas y se le pidió que lo adaptara para, en un primer caso, comparar KMeans con GMM, y en segundo lugar, para comparar entre distintos números de clústeres en KMeans. Más adelante se detallarán los resultados obtenidos en la implementación y la implicación de estos datos.

En la siguiente Figura, se muestran los resultados del script en una gráfica para el caso de generar 12 clústeres (nclusters) en KMeans e intentar formar los datos existentes con 12 distribuciones normales (ncomponents) con GMM. Al ser los resultados de KMeans mejores en todas las métricas, es la opción escogida para hacer las recomendaciones.

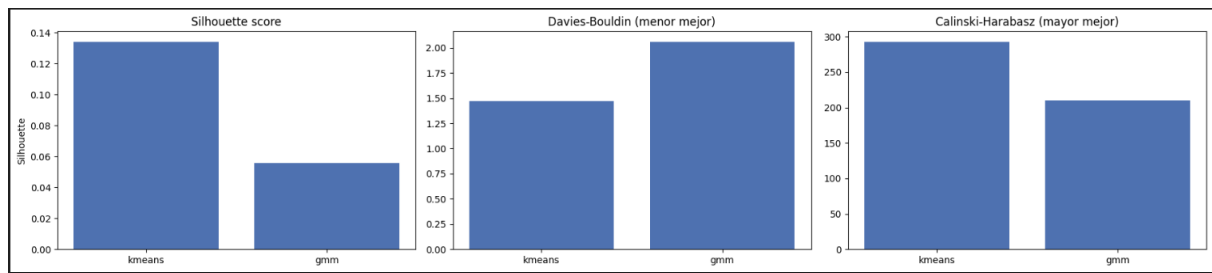


Figura 10: Comparación de medidas para los casos nclusters=ncomponents=12.

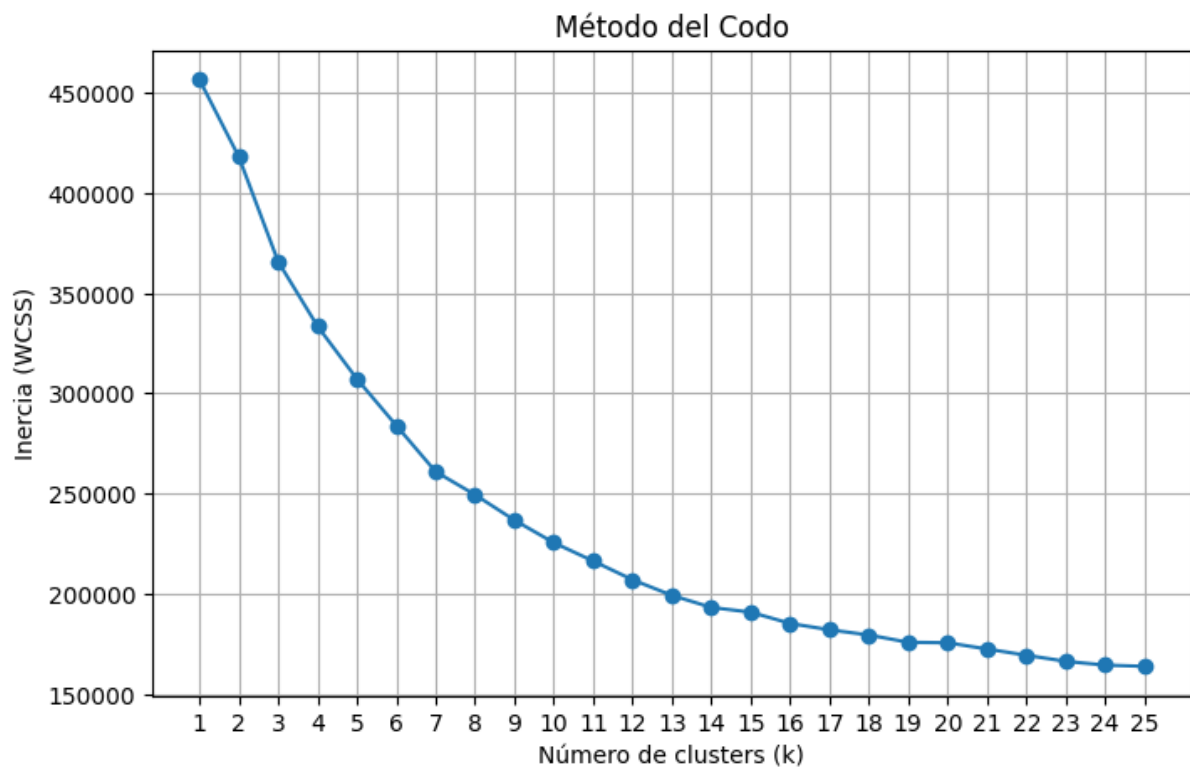


Figura 11: Método del codo para KMeans desde 1 a 25 clusters.

Como se observa en la Figura 10, el Silhouette Score toma un valor cercano al 0, lo cual nos indica que, aunque se diferencian los clústeres entre sí, existe solapamiento entre ellos. Por lo tanto, se ha tomado la decisión de reducir el número de clústeres, ya que como se observa en la Figura 11, no hay ningún punto especialmente bueno en el que poder fijarse para reducir el solapamiento entre clusters en valores intermedios, y en valores muy altos ($k > 15$), las medidas usadas nos indican que el solapamiento entre clústeres es aún mayor, lo cual puede llevar a predicciones erróneas por fallos de asignación.

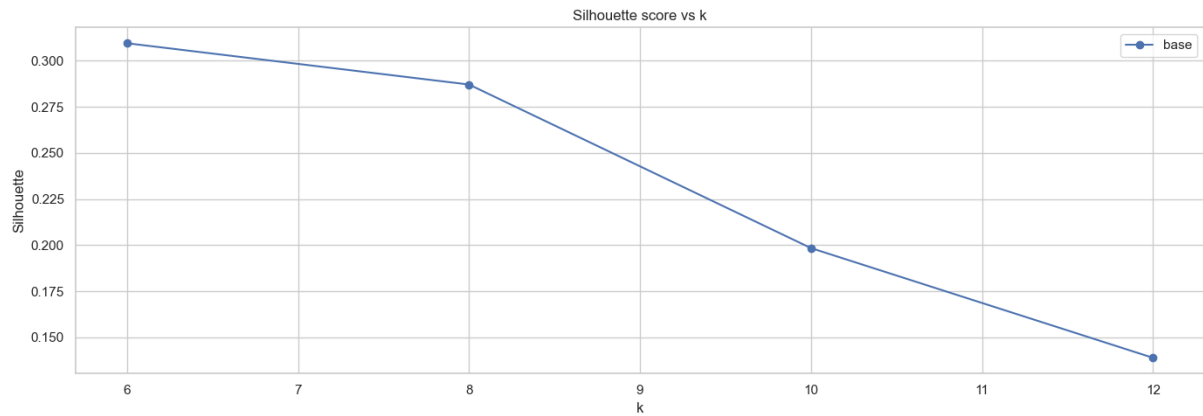


Figura 12: Cambio del Silhouette Score entre nclusters=6 y nclusters=12

En la Figura 12 se ve como el Silhouette score disminuye al aumentar el número de clústeres. Esto indica que no se necesitan tantos como estábamos probando inicialmente. Por lo que se ha llegado a la decisión final de usar un KMeans con $k=8$ clústeres, por su buen resultado en las métricas comparándolo con otras cantidades.

4.3.3. Evaluación de rendimiento

Como se ha indicado en la subsección Preprocesado de datos, para comparar la calidad de KMeans y GMM se ha evaluado la calidad de las agrupaciones y la estructura de los perfiles obtenidos mediante tres índices de validación externos/internos ampliamente usados en la literatura: Silhouette Score, Davies–Bouldin Index y Calinski–Harabasz Index [10], [15]–[17]. A continuación, los resultados para $nclusters = 8$, que ha acabado siendo la cantidad usada.

- **Silhouette score:** 0.285
- **Davies–Bouldin Index:** 1.581
- **Calinski–Harabasz Index:** 329.035

```
[RUN] KMeans (k=8)
Metrics KMeans: {'silhouette': 0.2870693017849553, 'davies_bouldin': 1.5002896080537744, 'calinski_harabasz': 348.56791006812733}
```

Figura 13: Resultado en Jupyter Notebook del script.

Se profundizará en detalle de las implicaciones de esta información en la sección de Resultados, pero nos hace entender que, aunque se encuentra separación entre los clusters, existe cierto solapamiento.

4.4. Frontend en Angular

4.4.1. Estructura de módulos y componentes

El proyecto Angular se organiza en:

- `app/alumnos/`: contiene todo lo relacionado con el uso del alumno de la página.
- `app/auth/`: implementa la autenticación de administrador, así como el funcionamiento del panel de éste.

Dentro de esta división, cada una de las secciones está compuesta por:

- `components`: partes reutilizables de la aplicación
- `pages`: las distintas páginas que conforman la aplicación
- `services`: realiza la comunicación con el backend
- `pipes`: modifican visualmente la aplicación web sin realizar cambios en los datos
- `interfaces`: modelan los tipos de datos que se utilizan

4.4.2. Páginas de la aplicación y capturas de interfaz

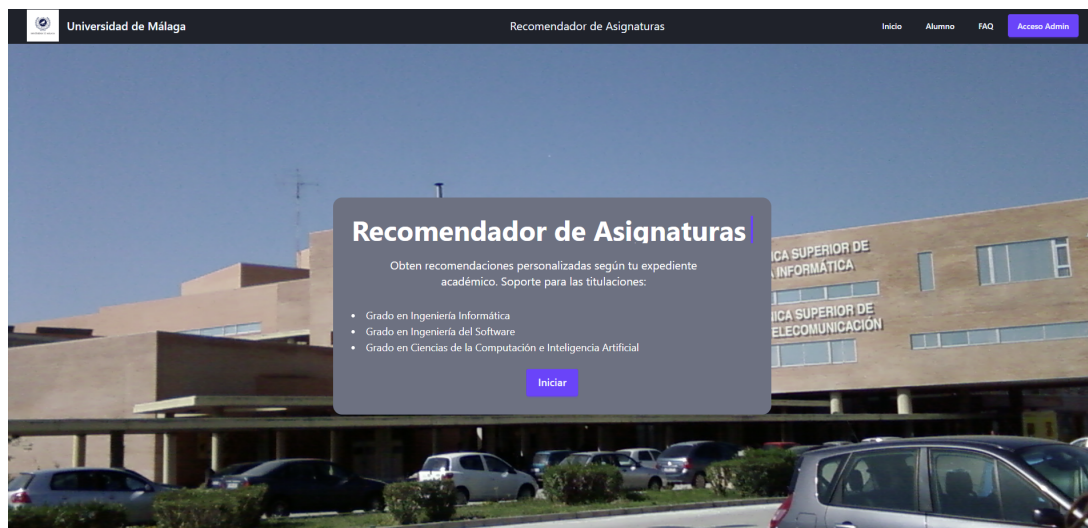


Figura 14: Vista inicial de la aplicación web.

Esta es la *landing page* de la aplicación, será a la que accedan todos los usuarios de inicio. Tiene un botón de **Iniciar** para los alumnos, y una barra de navegación común a todas las páginas, que permite una redirección rápida a cualquiera de las demás páginas, incluido

el Login para administradores. Es importante destacar que el *Login admin*, en caso de estar registrado como administrador cambia a **Panel admin**, o en caso de estar registrado como alumno cambia a **Mi perfil**, y pasa a funcionar igual que el botón **Alumno**, también en la barra de navegación.

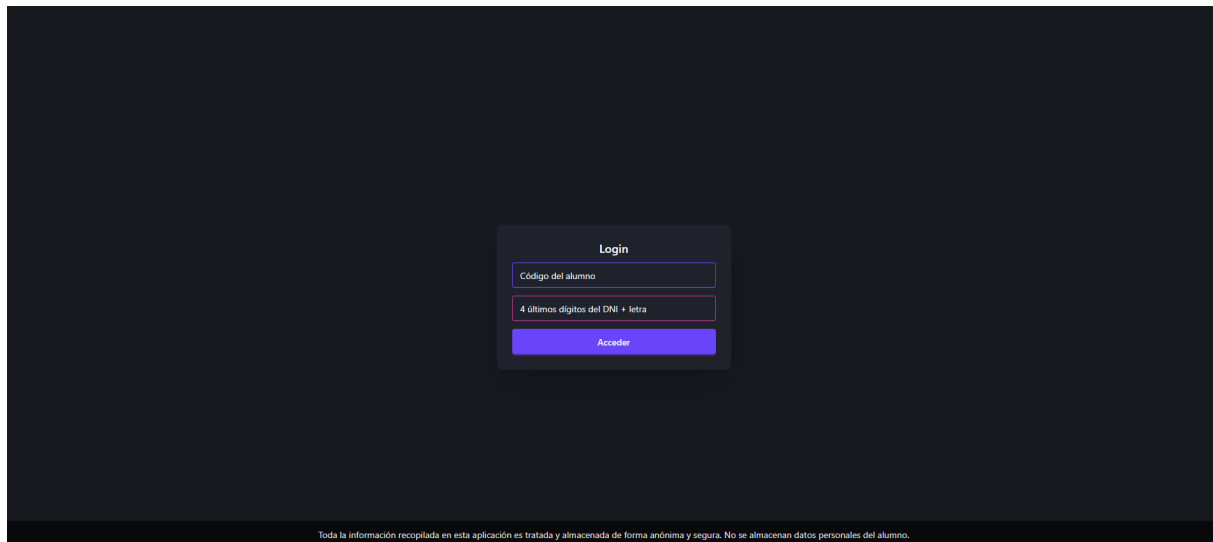
The image shows a dark-themed login form centered on a black background. The form is titled "Login" in white. It contains two input fields: the first is labeled "Código del alumno" and the second is labeled "4 últimos dígitos del DNI + letra". Below these fields is a blue button labeled "Acceder". At the bottom of the page, there is a small white text disclaimer: "Toda la información recopilada en esta aplicación es tratada y almacenada de forma anónima y segura. No se almacenan datos personales del alumno."

Figura 15: Página de Login/Creación de usuario para el alumno

Aquí el alumno introduce su número de expediente y su DNI, y la página consulta si está registrado, y verifica sus datos, o si no lo está, y le redirecciona a subir su expediente académico.

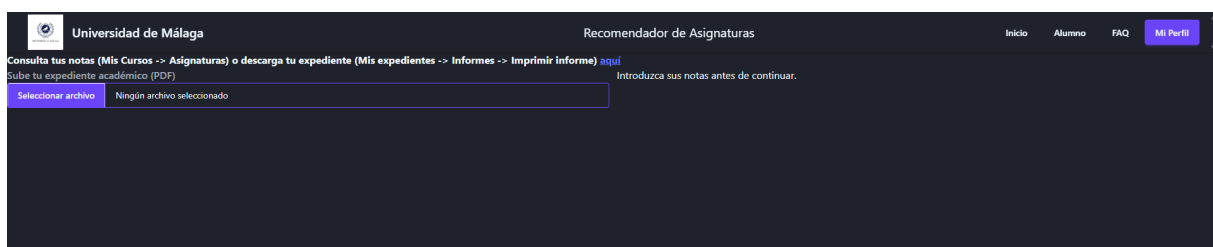
The image shows a web interface for the University of Málaga. The header includes the university logo, the name "Universidad de Málaga", and navigation links: "Inicio", "Alumno", "FAQ", and "Mi Perfil". The main content area has a title "Recomendador de Asignaturas". Below the title, there are links for "Consulta tus notas (Mis Cursos -> Asignaturas) o descarga tu expediente (Mis expedientes -> Informes -> Imprimir informe) aquí" and "Sube tu expediente académico (PDF)". There is a text input field for "Introduce tus notas antes de continuar." and a blue button labeled "Seleccionar archivo". Below the button, it says "Ningún archivo seleccionado".

Figura 16: Página para subir el expediente del alumno en caso de no estar guardado o querer actualizarlo

En esta página se envía el expediente académico al backend. También se asegura de que el nombre del documento sea correcto, sino salta un mensaje de error.

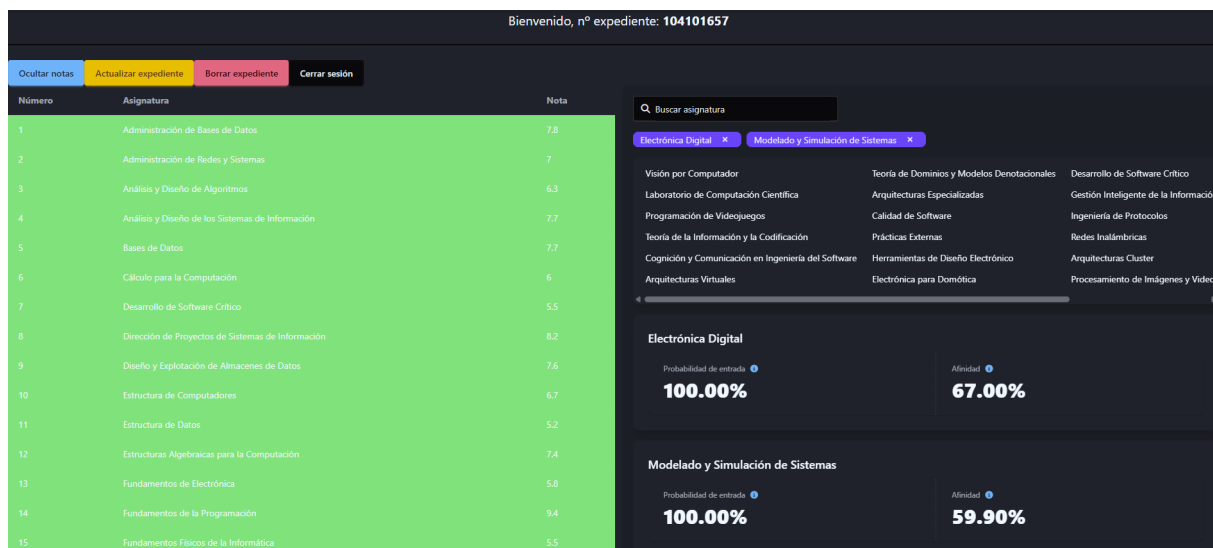


Figura 17: Vista de los datos de un alumno tras haber generado varias recomendaciones.

Esta es la página de un alumno después de que haya realizado varias consultas en la página. Como se ve, todas las optativas se le muestran en una lista, puede seleccionar todas las que quiera, y cuando se terminan los cálculos, se muestran en las distintas tarjetas. El alumno puede buscar una optativa por nombre con la barra de búsqueda de la que dispone, quitar la tarjeta de una optativa pulsando en el botón con el nombre de la optativa justo debajo de la barra de búsqueda o, en el lado izquierdo de la pantalla encima de las calificaciones, dispone del botón azul, que oculta o muestra las calificaciones según su estado

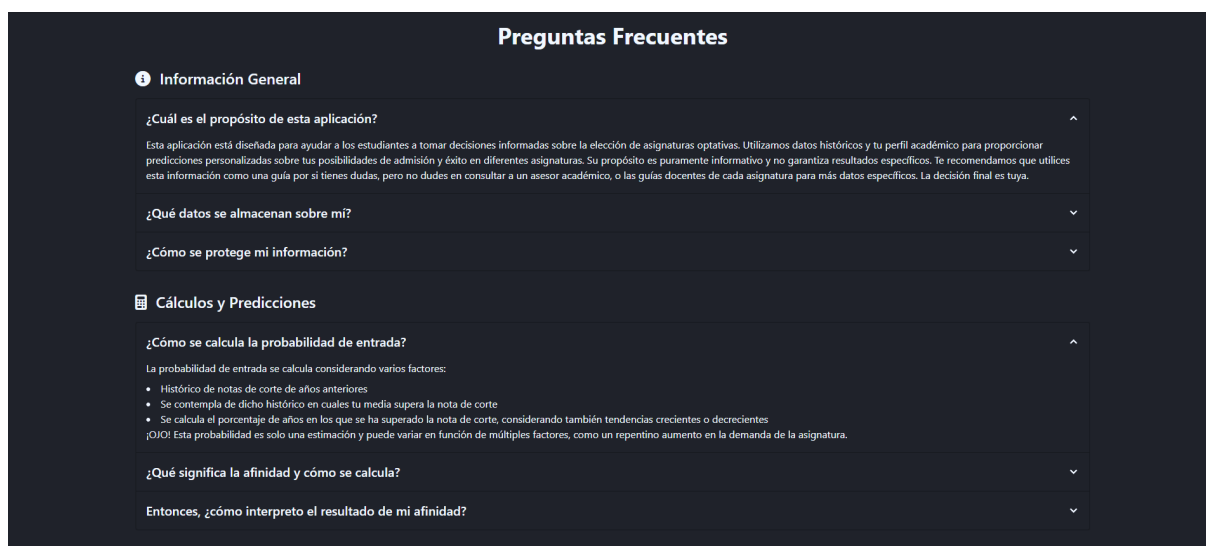


Figura 18: Página de preguntas frecuentes

En la página de preguntas frecuentes se han redactado ciertos puntos que pueden llegar a ser de interés para el alumno o que se han considerado necesarios que estén expresados por escrito, como los datos que se almacenan, cómo se protegen éstos o como se calculan las métricas de la aplicación.

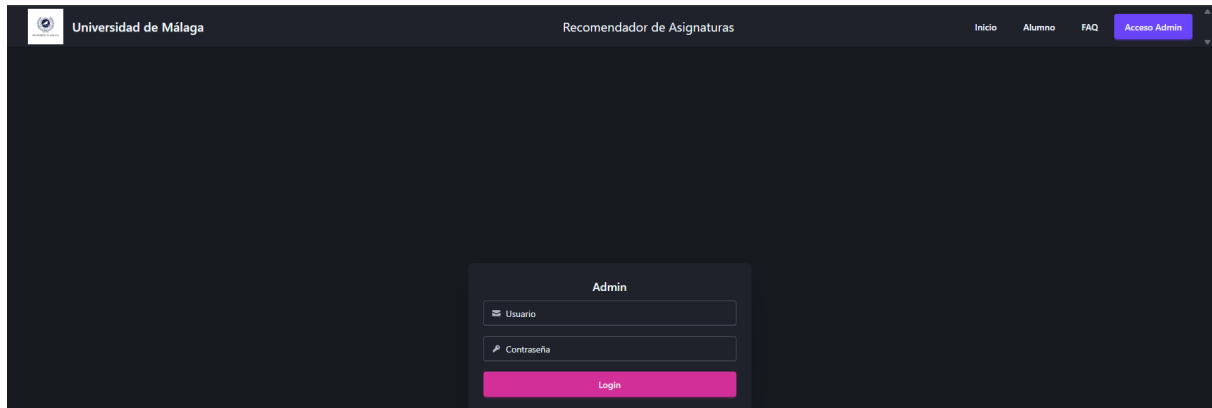


Figura 19: Página de login para administradores

Este es el login para administradores. En lugar de funcionar con número de expediente y DNI, funciona con usuario y contraseña. Si el usuario está registrado en la base de datos Oracle como administrador, se le da acceso al panel de administrador.

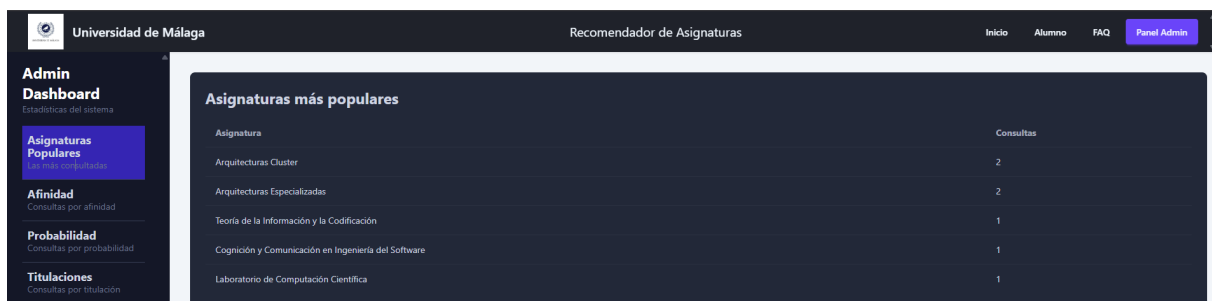


Figura 20: Panel de administrador

Este es el panel de administrador, el cual muestra las distintas métricas que se almacenan sobre las consultas de los alumnos, así como opciones para reiniciar los clústeres y modificar la ponderación de los distintos años a la hora de calcular la probabilidad de acceso.

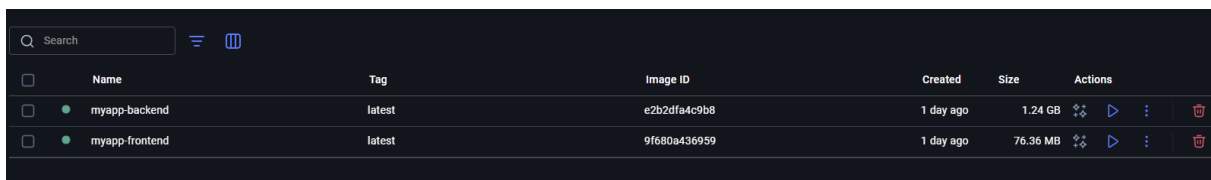
4.5. Contenerización e integración con Docker

La contenerización con Docker facilita la reproducibilidad del entorno de ejecución, el aislamiento de dependencias y el despliegue consistente entre entornos de desarrollo, prue-

bas y producción. En este trabajo se ha optado por empaquetar la aplicación en dos servicios principales: *frontend* (Angular servido por Nginx) y *backend* (FastAPI con Uvicorn/Gunicorn) orquestados mediante `docker-compose`.

Para implementarlos, se ha creado un fichero `docker-compose.yml` en la raíz del proyecto, que es el que ejecuta el resto de ficheros Dockerfile. Dentro de las carpetas de cada proyecto, hay un fichero `Dockerfile`, que contiene las instrucciones para crear los contenedores y que tengan instaladas las dependencias necesarias para que funcione. En el frontend también se añade un fichero `nginx.conf`, que contiene la configuración del servicio Nginx donde se despliega la aplicación.

Como resultado, se ha obtenido una implementación funcional de la aplicación mediante Docker, lo que permite la ejecución de la aplicación sin necesidad de instalar las aplicaciones en la máquina local.



☐	Name	Tag	Image ID	Created	Size	Actions
☐	myapp-backend	latest	e2b2dfa4c9b8	1 day ago	1.24 GB	
☐	myapp-frontend	latest	9f680a436959	1 day ago	76.36 MB	

Figura 21: Captura de Docker Desktop con ambos contenedores ejecutandose

4.6. Protección de datos

Para garantizar el cumplimiento de la Ley Orgánica de Protección de Datos, se realizan las siguientes medidas:

- El acceso a los datos del alumno está protegido por el número de expediente y los 4 últimos dígitos del DNI + contraseña. Solo el alumno que es el que posee estos datos puede acceder a ellos.
- El almacenamiento de los datos usados se guarda temporalmente en el SessionStorage y no en el LocalStorage. Esto significa que, al cerrar la pestaña donde se abre la aplicación, se borran también todos los datos que usa la aplicación para agilizar procesos.
- A la hora de almacenar datos sensibles del alumno, como su DNI, se evita usarlos completos, tomando solo la parte final del DNI, y codificandola mediante SHA-256 para mayor protección. De esta manera no solo los datos almacenados sobre el alumno son los mínimos necesarios, sino que además datos que puedan identificarle no se almacenan como texto plano.

- Control de acceso a distintas páginas controlado por rol y login. Para acceder a la página de un alumno concreto debes identificarte con sus credenciales, y para acceder a las páginas de admin debes acceder al login de admin y entrar con un usuario y una contraseña que esté reconocido como administrador.
- Los datos que se almacenan del alumno están indicados en la página de preguntas frecuentes, así como el motivo por el que se guardan, para que quede constancia de que todos los datos introducidos y almacenados son relevantes e imprescindibles.
- El alumno puede ver los datos de su expediente que se guardan en todo momento, así como borrarlos y modificarlos. Al eliminar el expediente, también se elimina de la base de datos su registro en la tabla **ALUMNOS_VERIFICADOS**, así que tampoco quedan guardados su número de expediente y DNI.
- Se añade una nota legal en el footer de la página para introducir los datos como alumno indicando el consentimiento y cómo pueden borrarse los datos almacenados en cualquier momento. Así mismo, en la página de preguntas frecuentes, se detallan qué datos se guardan para que el alumno los sepa en todo momento.

5

Pruebas y validación

5.1. Estrategia de pruebas

Para garantizar la calidad del sistema se ha definido una estrategia en tres niveles:

- **Pruebas unitarias:** verificación aislada de funciones y clases en el backend. En el frontend se realizaron comprobaciones manuales y tests unitarios de distintos componentes y servicios.
- **Pruebas de integración:** validación de la interacción entre módulos (API–modelo ML, frontend–backend).
- **Pruebas de sistema:** evaluación del flujo completo de usuario bajo condiciones reales.

5.2. Herramientas

- **pytest:** ejecución de tests unitarios para el backend Python.
- **Jasmine y Karma:** ejecución de tests unitarios para el frontend Angular.
- **Postman:** pruebas manuales de los endpoints REST.
- **Navegador/DevTools:** verificación manual del frontend Angular (renderizado y eventos).

5.3. Plan de pruebas

5.3.1. Pruebas de uso

Se han definido casos de prueba que cubren las funcionalidades críticas:

- **Carga de expediente:** comprobación de subida y parseo de PDF.
- **Preprocesado de datos:** validación de limpieza, normalización y detección de valores nulos.

- **Modelo de recomendación:** generación de recomendaciones y cálculo de probabilidades.
- **API REST:** respuesta correcta y código HTTP según distintos escenarios.
- **Interfaz de usuario:** renderizado de resultados y manejo de errores (pruebas manuales).

Para realizar las pruebas manuales en el sistema, se ha contado con la colaboración de un compañero estudiante en la Escuela Técnica Superior del Grado en Ingeniería Informática del plan de 2010, el cual ha consentido el uso de sus datos como usuario de prueba. Aun así, se borrarán sus datos del sistema tras finalizar el proceso por privacidad. Además, se añadirá de forma manual un alumno del Grado en Ingeniería del Software del plan de 2023.

Realizando pruebas de uso de la aplicación, se puede observar que:

- La protección mediante número de expediente y parte final del DNI es funcional. Los distintos usuarios solo pueden acceder con sus credenciales correctas, y se diferencian entre ellas.
- Para cada uno de los alumnos, se carga su página correspondiente:

Bienvenido, nº expediente: 104101659		
Ocultar notas Actualizar expediente Borrar expediente Cerrar sesión		
Número	Asignatura	Nota
1	Administración de Bases de Datos	3.1
2	Análisis y Diseño de Algoritmos	3
3	Análisis y Diseño de los Sistemas de Información	7.7
4	Arquitecturas Especializadas	0
5	Bases de Datos	5.4
6	Cálculo para la Computación	5
7	Estructura de Computadores	5
8	Estructura de Datos	5.5
9	Estructuras Algebraicas para la Computación	5.7
10	Fundamentos de Electrónica	5.3

- Visión por Computador
- Laboratorio de Computación Científica
- Programación de Videojuegos
- Teoría de la Información y la Codificación
- Cognición y Comunicación en Ingeniería del Software
- Arquitecturas Virtuales
- Teoría de Dominios y Modelos Denotacionales
- Arquitecturas Especializadas
- Calidad de Software
- Prácticas Externas
- Electrónica Digital
- Herramientas de Diseño Electrónico
- Electrónica para Domótica
- Desarrollo de Software Crítico
- Gestión Inteligente de la Información
- Ingeniería de Protocolos
- Redes Inalámbricas
- Arquitecturas Cluster

Figura 22: Página del alumno de Ingeniería Informática

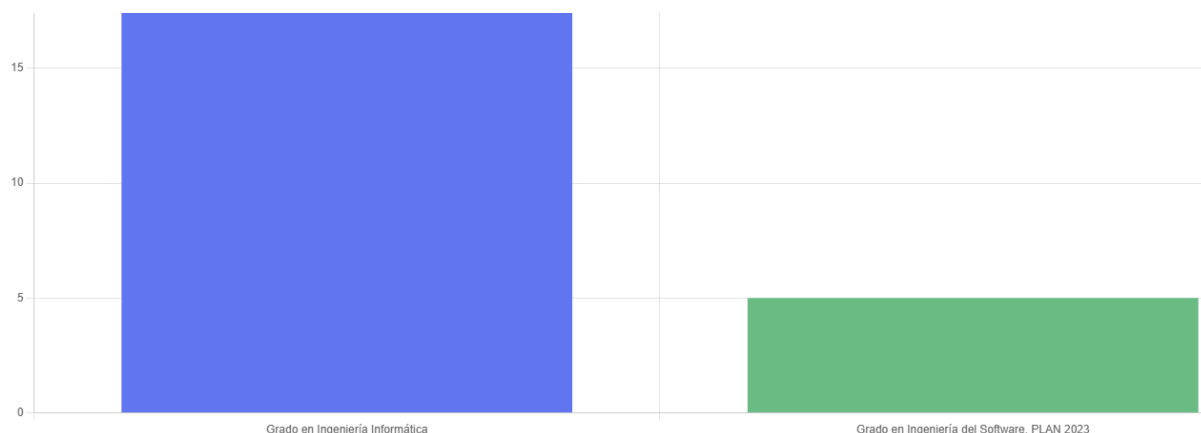


Figura 26: Página de titulaciones tras realizar consultas con un alumno de una titulación distinta

Realizando las pruebas, también se ha encontrado un fallo

5.3.2. Pruebas unitarias

Para cumplir con el proceso de ingeniería del software, y para demostrar en el ámbito de este Trabajo de Fin de Grado el conocimiento sobre la implementación de pruebas unitarias, se han realizado los siguientes procedimientos.

En el frontend, se han implementado con Jasmine y Karma una serie de ficheros de prueba (extension **.spec.ts**) para comprobar el correcto comportamiento independiente de los distintos componentes y servicios de la aplicación, consiguiendo una cobertura completa de código para los componentes testeados.

app/alumnos/components/navbar		100%	14/14	100%	2/2	100%	4/4	100%	13/13
app/alumnos/components/tarea-notas		100%	2/2	100%	0/0	100%	0/0	100%	2/2
app/alumnos/pages/alumno-page		100%	23/23	100%	4/4	100%	9/9	100%	22/22
app/alumnos/pipes		100%	7/7	100%	1/1	100%	2/2	100%	5/5
app/alumnos/services		100%	36/36	100%	0/0	100%	16/16	100%	33/33

Figura 27: Cobertura de distintos componentes del frontend en Karma

En el backend, se han comprobado todos los endpoints de la aplicación usando pytest, consiguiendo una cobertura total del código de un 82 %, comprobando todas las rutas y todos los endpoints, faltando solamente la comprobación individual de algunas funciones del fichero funciones.py que, al comprobarse las rutas, se puede confirmar también su correcto funcionamiento ya que éstas hacen uso del fichero de funciones.

Name	Stmts	Miss	Cover	Missing
app__init__.py	0	0	100%	
app\funciones.py	354	183	48%	36-42, 43-49, 50-56, 57-63, 64-70, 71-77, 78-84, 85-91, 92-98, 99-105, 106-112, 113-119, 120-126, 127-133, 134-140, 141-147, 148-154, 155-161, 162-168, 169-175, 176-182, 183-189, 190-196, 197-203, 204-210, 211-217, 218-224, 225-231, 232-238, 239-245, 246-252, 253-259, 260-266, 267-273, 274-280, 281-287, 288-294, 295-301, 302-308, 309-315, 316-322, 323-329, 330-336, 337-343, 344-350, 351-357, 358-364, 365-371, 372-378, 379-385, 386-392, 393-399, 400-406, 407-413, 414-420, 421-427, 428-434, 435-441, 442-448, 449-455, 456-462, 463-469, 470-476, 477-483, 484-490, 491-497, 498-504, 505-511, 512-518, 519-525, 526-532, 533-539, 540-546, 547-553, 554-560, 561-567, 568-574, 575-581, 582-588, 589-595, 596-602, 603-609, 610-616, 617-623, 624-630, 631-637, 638-644, 645-651, 652-658, 659-665, 666-672, 673-679, 680-686, 687-693, 694-700, 701-707, 708-714, 715-721, 722-728, 729-735, 736-742, 743-749, 750-756, 757-763, 764-770, 771-777, 778-784, 785-791, 792-798, 799-805, 806-812, 813-819, 820-826, 827-833, 834-840, 841-847, 848-854, 855-861, 862-868, 869-875, 876-882, 883-889, 890-896, 897-903, 904-910, 911-917, 918-924, 925-931, 932-938, 939-945, 946-952, 953-959, 960-966, 967-973, 974-980, 981-987, 988-994, 995-1001, 1002-1008, 1009-1015, 1016-1022, 1023-1029, 1030-1036, 1037-1043, 1044-1050, 1051-1057, 1058-1064, 1065-1071, 1072-1078, 1079-1085, 1086-1092, 1093-1099, 1100-1106, 1107-1113, 1114-1120, 1121-1127, 1128-1134, 1135-1141, 1142-1148, 1149-1155, 1156-1162, 1163-1169, 1170-1176, 1177-1183, 1184-1190, 1191-1197, 1198-1204, 1205-1211, 1212-1218, 1219-1225, 1226-1232, 1233-1239, 1240-1246, 1247-1253, 1254-1260, 1261-1267, 1268-1274, 1275-1281, 1282-1288, 1289-1295, 1296-1302, 1303-1309, 1310-1316, 1317-1323, 1324-1330, 1331-1337, 1338-1344, 1345-1351, 1352-1358, 1359-1365, 1366-1372, 1373-1379, 1380-1386, 1387-1393, 1394-1400, 1401-1407, 1408-1414, 1415-1421, 1422-1428, 1429-1435, 1436-1442, 1443-1449, 1450-1456, 1457-1463, 1464-1470, 1471-1477, 1478-1484, 1485-1491, 1492-1498, 1499-1505, 1506-1512, 1513-1519, 1520-1526, 1527-1533, 1534-1540, 1541-1547, 1548-1554, 1555-1561, 1562-1568, 1569-1575, 1576-1582, 1583-1589, 1590-1596, 1597-1603, 1604-1610, 1611-1617, 1618-1624, 1625-1631, 1632-1638, 1639-1645, 1646-1652, 1653-1659, 1660-1666, 1667-1673, 1674-1680, 1681-1687, 1688-1694, 1695-1701, 1702-1708, 1709-1715, 1716-1722, 1723-1729, 1730-1736, 1737-1743, 1744-1750, 1751-1757, 1758-1764, 1765-1771, 1772-1778, 1779-1785, 1786-1792, 1793-1799, 1800-1806, 1807-1813, 1814-1820, 1821-1827, 1828-1834, 1835-1841, 1842-1848, 1849-1855, 1856-1862, 1863-1869, 1870-1876, 1877-1883, 1884-1890, 1891-1897, 1898-1904, 1905-1911, 1912-1918, 1919-1925, 1926-1932, 1933-1939, 1940-1946, 1947-1953, 1954-1960, 1961-1967, 1968-1974, 1975-1981, 1982-1988, 1989-1995, 1996-2002, 2003-2009, 2010-2016, 2017-2023, 2024-2030, 2031-2037, 2038-2044, 2045-2051, 2052-2058, 2059-2065, 2066-2072, 2073-2079, 2080-2086, 2087-2093, 2094-2100, 2101-2107, 2108-2114, 2115-2121, 2122-2128, 2129-2135, 2136-2142, 2143-2149, 2150-2156, 2157-2163, 2164-2170, 2171-2177, 2178-2184, 2185-2191, 2192-2198, 2199-2205, 2206-2212, 2213-2219, 2220-2226, 2227-2233, 2234-2240, 2241-2247, 2248-2254, 2255-2261, 2262-2268, 2269-2275, 2276-2282, 2283-2289, 2290-2296, 2297-2303, 2304-2310, 2311-2317, 2318-2324, 2325-2331, 2332-2338, 2339-2345, 2346-2352, 2353-2359, 2360-2366, 2367-2373, 2374-2380, 2381-2387, 2388-2394, 2395-2401, 2402-2408, 2409-2415, 2416-2422, 2423-2429, 2430-2436, 2437-2443, 2444-2450, 2451-2457, 2458-2464, 2465-2471, 2472-2478, 2479-2485, 2486-2492, 2493-2499, 2500-2506, 2507-2513, 2514-2520, 2521-2527, 2528-2534, 2535-2541, 2542-2548, 2549-2555, 2556-2562, 2563-2569, 2570-2576, 2577-2583, 2584-2590, 2591-2597, 2598-2604, 2605-2611, 2612-2618, 2619-2625, 2626-2632, 2633-2639, 2640-2646, 2647-2653, 2654-2660, 2661-2667, 2668-2674, 2675-2681, 2682-2688, 2689-2695, 2696-2702, 2703-2709, 2710-2716, 2717-2723, 2724-2730, 2731-2737, 2738-2744, 2745-2751, 2752-2758, 2759-2765, 2766-2772, 2773-2779, 2780-2786, 2787-2793, 2794-2800, 2801-2807, 2808-2814, 2815-2821, 2822-2828, 2829-2835, 2836-2842, 2843-2849, 2850-2856, 2857-2863, 2864-2870, 2871-2877, 2878-2884, 2885-2891, 2892-2898, 2899-2905, 2906-2912, 2913-2919, 2920-2926, 2927-2933, 2934-2940, 2941-2947, 2948-2954, 2955-2961, 2962-2968, 2969-2975, 2976-2982, 2983-2989, 2990-2996, 2997-3003, 3004-3010, 3011-3017, 3018-3024, 3025-3031, 3032-3038, 3039-3045, 3046-3052, 3053-3059, 3060-3066, 3067-3073, 3074-3080, 3081-3087, 3088-3094, 3095-3101, 3102-3108, 3109-3115, 3116-3122, 3123-3129, 3130-3136, 3137-3143, 3144-3150, 3151-3157, 3158-3164, 3165-3171, 3172-3178, 3179-3185, 3186-3192, 3193-3199, 3200-3206, 3207-3213, 3214-3220, 3221-3227, 3228-3234, 3235-3241, 3242-3248, 3249-3255, 3256-3262, 3263-3269, 3270-3276, 3277-3283, 3284-3290, 3291-3297, 3298-3304, 3305-3311, 3312-3318, 3319-3325, 3326-3332, 3333-3339, 3340-3346, 3347-3353, 3354-3360, 3361-3367, 3368-3374, 3375-3381, 3382-3388, 3389-3395, 3396-3402, 3403-3409, 3410-3416, 3417-3423, 3424-3430, 3431-3437, 3438-3444, 3445-3451, 3452-3458, 3459-3465, 3466-3472, 3473-3479, 3480-3486, 3487-3493, 3494-3500, 3501-3507, 3508-3514, 3515-3521, 3522-3528, 3529-3535, 3536-3542, 3543-3549, 3550-3556, 3557-3563, 3564-3570, 3571-3577, 3578-3584, 3585-3591, 3592-3598, 3599-3605, 3606-3612, 3613-3619, 3620-3626, 3627-3633, 3634-3640, 3641-3647, 3648-3654, 3655-3661, 3662-3668, 3669-3675, 3676-3682, 3683-3689, 3690-3696, 3697-3703, 3704-3710, 3711-3717, 3718-3724, 3725-3731, 3732-3738, 3739-3745, 3746-3752, 3753-3759, 3760-3766, 3767-3773, 3774-3780, 3781-3787, 3788-3794, 3795-3801, 3802-3808, 3809-3815, 3816-3822, 3823-3829, 3830-3836, 3837-3843, 3844-3850, 3851-3857, 3858-3864, 3865-3871, 3872-3878, 3879-3885, 3886-3892, 3893-3899, 3900-3906, 3907-3913, 3914-3920, 3921-3927, 3928-3934, 3935-3941, 3942-3948, 3949-3955, 3956-3962, 3963-3969, 3970-3976, 3977-3983, 3984-3990, 3991-3997, 3998-4004, 4005-4011, 4012-4018, 4019-4025, 4026-4032, 4033-4039, 4040-4046, 4047-4053, 4054-4060, 4061-4067, 4068-4074, 4075-4081, 4082-4088, 4089-4095, 4096-4102, 4103-4109, 4110-4116, 4117-4123, 4124-4130, 4131-4137, 4138-4144, 4145-4151, 4152-4158, 4159-4165, 4166-4172, 4173-4179, 4180-4186, 4187-4193, 4194-4200, 4201-4207, 4208-4214, 4215-4221, 4222-4228, 4229-4235, 4236-4242, 4243-4249, 4250-4256, 4257-4263, 4264-4270, 4271-4277, 4278-4284, 4285-4291, 4292-4298, 4299-4305, 4306-4312, 4313-4319, 4320-4326, 4327-4333, 4334-4340, 4341-4347, 4348-4354, 4355-4361, 4362-4368, 4369-4375, 4376-4382, 4383-4389, 4390-4396, 4397-4403, 4404-4410, 4411-4417, 4418-4424, 4425-4431, 4432-4438, 4439-4445, 4446-4452, 4453-4459, 4460-4466, 4467-4473, 4474-4480, 4481-4487, 4488-4494, 4495-4501, 4502-4508, 4509-4515, 4516-4522, 4523-4529, 4530-4536, 4537-4543, 4544-4550, 4551-4557, 4558-4564, 4565-4571, 4572-4578, 4579-4585, 4586-4592, 4593-4599, 4600-4606, 4607-4613, 4614-4620, 4621-4627, 4628-4634, 4635-4641, 4642-4648, 4649-4655, 4656-4662, 4663-4669, 4670-4676, 4677-4683, 4684-4690, 4691-4697, 4698-4704, 4705-4711, 4712-4718, 4719-4725, 4726-4732, 4733-4739, 4740-4746, 4747-4753, 4754-4760, 4761-4767, 4768-4774, 4775-4781, 4782-4788, 4789-4795, 4796-4802, 4803-4809, 4810-4816, 4817-4823, 4824-4830, 4831-4837, 4838-4844, 4845-4851, 4852-4858, 4859-4865, 4866-4872, 4873-4879, 4880-4886, 4887-4893, 4894-4900, 4901-4907, 4908-4914, 4915-4921, 4922-4928, 4929-4935, 4936-4942, 4943-4949, 4950-4956, 4957-4963, 4964-4970, 4971-4977, 4978-4984, 4985-4991, 4992-4998, 4999-5005, 5006-5012, 5013-5019, 5020-5026, 5027-5033, 5034-5040, 5041-5047, 5048-5054, 5055-5061, 5062-5068, 5069-5075, 5076-5082, 5083-5089, 5090-5096, 5097-5103, 5104-5110, 5111-5117, 5118-5124, 5125-5131, 5132-5138, 5139-5145, 5146-5152, 5153-5159, 5160-5166, 5167-5173, 5174-5180, 5181-5187, 5188-5194, 5195-5201, 5202-5208, 5209-5215, 5216-5222, 5223-5229, 5230-5236, 5237-5243, 5244-5250, 5251-5257, 5258-5264, 5265-5271, 5272-5278, 5279-5285, 5286-5292, 5293-5299, 5300-5306, 5307-5313, 5314-5320, 5321-5327, 5328-5334, 5335-5341, 5342-5348, 5349-5355, 5356-5362, 5363-5369, 5370-5376, 5377-5383, 5384-5390, 5391-5397, 5398-5404, 5405-5411, 5412-5418, 5419-5425, 5426-5432, 5433-5439, 5440-5446, 5447-5453, 5454-5460, 5461-5467, 5468-5474, 5475-5481, 5482-5488, 5489-5495, 5496-5502, 5503-5509, 5510-5516, 5517-5523, 5524-5530, 5531-5537, 5538-5544, 5545-5551, 5552-5558, 5559-5565, 5566-5572, 5573-5579, 5580-5586, 5587-5593, 5594-5600, 5601-5607, 5608-5614, 5615-5621, 5622-5628, 5629-5635, 5636-5642, 5643-5649, 5650-5656, 5657-5663, 5664-5670, 5671-5677, 5678-5684, 5685-5691, 5692-5698, 5699-5705, 5706-5712, 5713-5719, 5720-5726, 5727-5733, 5734-5740, 5741-5747, 5748-5754, 5755-5761, 5762-5768, 5769-5775, 5776-5782, 5783-5789, 5790-5796, 5797-5803, 5804-5810, 5811-5817, 5818-5824, 5825-5831, 5832-5838, 5839-5845, 5846-5852, 5853-5859, 5860-5866, 5867-5873, 5874-5880, 5881-5887, 5888-5894, 5895-5901, 5902-5908, 5909-5915, 5916-5922, 5923-5929, 5930-5936, 5937-5943, 5944-5950, 5951-5957, 5958-5964, 5965-5971, 5972-5978, 5979-5985, 5986-5992, 5993-5999, 6000-6006, 6007-6013, 6014-6020, 6021-6027, 6028-6034, 6035-6041, 6042-6048, 6049-6055, 6056-6062, 6063-6069, 6070-6076, 6077-6083, 6084-6090, 6091-6097, 6098-6104, 6105-6111, 6112-6118, 6119-6125, 6126-6132, 6133-6139, 6140-6146, 6147-6153, 6154-6160, 6161-6167, 6168-6174, 6175-6181, 6182-6188, 6189-6195, 6196-6202, 6203-6209, 6210-6216, 6217-6223, 6224-6230, 6231-6237, 6238-6244, 6245-6251, 6252-6258, 6259-6265, 6266-6272, 6273-6279, 6280-6286, 6287-6293, 6294-6300, 6301-6307, 6308-6314, 6315-6321, 6322-6328, 6329-6335, 6336-6342, 6343-6349, 6350-6356, 6357-6363, 6364-6370, 6371-6377, 6378-6384, 6385-6391, 6392-6398, 6399-6405, 6406-6412, 6413-6419, 6420-6426, 6427-6433, 6434-6440, 6441-6447, 6448-6454, 6455-6461, 6462-6468, 6469-6475, 6476-6482, 6483-6489, 6490-6496, 6497-6503, 6504-6510, 6511-6517, 6518-6524, 6525-6531, 6532-6538, 6539-6545, 6546-6552, 6553-6559, 6560-6566, 6567-6573, 6574-6580, 6581-6587, 6588-6594, 6595-6601, 6602-6608, 6609-6615, 6616-6622, 6623-6629, 6630-6636, 6637-6643, 6644-6650, 6651-6657, 6658-6664, 6665-6671, 6672-6678, 6679-6685, 6686-6692, 6693-6699, 6700-6706, 6707-6713, 6714-6720, 6721-6727, 6728-6734, 6735-6741, 6742-6748, 6749-6755, 6756-6762, 6763-6769, 6770-6776, 6777-6783, 6784-6790, 6791-6797, 6798-6804, 6805-6811, 6812-6818, 6819-6825, 6826-6832, 6833-6839, 6840-6846, 6847-6853, 6854-6860, 6861-6867, 6868-6874, 6875-6881, 6882-6888, 6889-6895, 6896-6902, 6903-6909, 6910-6916, 6917-6923, 6924-6930, 6931-6937, 6938-6944, 6945-6951, 6952-6958, 6959-6965, 6966-6972, 6973-6979, 6980-6986, 6987-6993, 6994-7000, 7001-7007, 7008-7014, 7015-7021, 7022-7028, 7029-7035, 7036-7042, 7043-7049, 7050-7056, 7057-7063, 7064-7070, 7071-7077, 7078-7084, 7085-7091, 7092-7098, 7099-7105, 7106-7112, 7113-7119, 7120-7126, 7127-7133, 7134-7140, 7141-7147, 7148-7154, 7155-7161, 7162-7168, 7169-7175, 7176-7182, 7183-7189, 7190-7196, 7197-7203, 7204-7210, 7211-7217, 7218-7224, 7225-7231, 7232-7238, 7239-7245, 7246-7252, 7253-7259, 7260-7266, 7267-7273, 7274-7280, 7281-7287, 7288-7294, 7295-7301, 7302-7308, 7309-7315, 7316-7322, 7323-7329, 7330-7336, 7337-7343, 7344-7350, 7351-7357, 7358-7364, 7365-7371, 7372-7378, 7379-7385, 7386-7392, 7393-7399, 7400-7406, 7407-7413, 7414-7420, 7421-7427, 7428-7434, 7435-7441, 7442-7448, 7449-7455, 7456-7462, 7463-7469, 7470-7476, 7477-7483, 7484-7490, 7491-7497, 7498-7504, 7505-7511, 7512-7518, 7519-7525, 7526-7532, 7533-7539, 7540-7546, 7547-7553, 7554-7560, 7561-7567, 7568-7574, 7575-7581, 7582-7588, 7589-7595, 7596-7602, 7603-7609, 7610-7616, 7617-7623, 7624-7630, 7631-7637, 7638-7644, 7645-7651, 7652-7658, 7659-7665, 7666-7672, 7673-7679, 7680-7686, 7687-7693, 7694-7700, 7701-7707, 7708-7714, 7715-7721, 7722-7728, 7729-7735, 7736-7742, 7743-7749, 7750-7756, 7757-7763, 7764-7770, 7771-7777, 7778-7784, 7785-7791, 7792-7798, 7799-7805, 7806-7812, 7813-7819, 7820-7826, 7827-7833, 7834-7840, 7841-7847, 7848-7854, 7855-7861, 7862-7868, 7869-7875, 7876-7882, 7883-7889, 7890-7896, 7897-7903, 7904-7910, 7911-7917, 7918-7924, 7925-7931, 7932-7938, 7939-7945, 7946-7952, 7953-7959, 7960-7966, 7967-7973, 7974-7980, 7981-7987, 7988-7994, 7995-8001, 8002-8008, 8009-

6

Resultados y discusión

6.1. Análisis de métricas de recomendación

A continuación se interpreta cada medida mencionada en la sección de Implementación y se discuten su significado y su implicación:

Silhouette score. El coeficiente de Silhouette toma valores en el intervalo $[-1, 1]$; valores próximos a 1 indican una buena separación entre clústeres, valores cercanos a 0 indican solapamiento y valores negativos mala asignación [15]. Un umbral orientativo habitual considera valores mayores a 0.5 como buenos, entre 0.25 y 0.5 como estructura débil-moderada, y por debajo de 0.25 como ausencia de estructura clara. El valor obtenido (0.285) se encuentra en la banda baja-moderada, lo que sugiere que existe cierta agrupación en los datos, pero con solapamientos relevantes entre grupos. Esto es esperable en dominios académicos reales (heterogeneidad de expedientes, asignaturas con perfiles mixtos) y no invalida la aproximación, sino que indica margen para mejorar la separabilidad mediante ingeniería de características o ajustes de modelos [7], [15]).

Davies–Bouldin Index (DBI). El índice DB cuantifica la ratio entre la dispersión intra-clúster y la separación inter-clúster; valores más bajos son mejores [16]. Un DBI de 1.581 puede considerarse moderado (no excelente, pero tampoco elevado), lo que concuerda con la conclusión anterior: los clústeres presentan una compacidad razonable pero no una separación nítida. En trabajos prácticos se observa que valores en torno a 1–2 aparecen con frecuencia en datos reales con ruido y características mixtas [16], [18].

Calinski–Harabasz Index (CHI). El índice CH aumenta con una mayor separación entre clústeres y compacidad interna; su escala depende del tamaño de la muestra y del número de clústeres [17]. Un valor absoluto (329.035) debe interpretarse relativo al contexto (número de instancias y clústeres). En nuestro caso, con el conjunto de validación utilizado (orden de

cientos de expedientes), dicho valor es coherente con una estructura de clústeres aceptable, corroborando la interpretación combinada de Silhouette y DBI [10], [17].

Evaluación global y implicaciones prácticas. Tomadas en conjunto, las tres métricas apuntan a una *estructura de clústeres moderada*: hay señales útiles para la recomendación (no es ruido total), pero la separación no es óptima. Esto es un resultado razonable para una primera versión del sistema basada en features extraídas automáticamente del expediente (notas, créditos, códigos), especialmente cuando el volumen de datos es limitado y las asignaturas presentan solapamiento competencial. La literatura recomienda combinar varias métricas y no fiarse de una sola medida; por ello, la evaluación multidimensional realizada está alineada con buenas prácticas [15]-[17], [19].

Defensa como primera aproximación y líneas de mejora. Considerando que este trabajo constituye una primera aproximación práctica al problema, los resultados se defienden porque:

- Proporcionan evidencia cuantitativa de que el modelo captura patrones relevantes del historial académico (Silhouette positivo y CH apreciable).
- Permiten ofrecer recomendaciones utilitarias al estudiante y estimaciones de probabilidad de plaza que ya aportan valor informativo frente al estado actual (gestión manual).
- Sirven como base empírica para iterar: con más datos y refinamientos es razonable esperar mejoras en las métricas y en la precisión percibida por usuarios.

6.2. Evaluación de usabilidad (pruebas manuales)

Dado que no fue posible realizar un estudio con usuarios externos voluntarios, la evaluación de usabilidad se realizó mediante pruebas manuales estructuradas llevadas a cabo por el autor. El objetivo de estas pruebas fue comprobar el correcto funcionamiento de la interfaz, la experiencia básica del usuario y la velocidad de respuesta del sistema en escenarios representativos de uso.

Procedimiento Se definieron una serie de casos de prueba orientados a las tareas críticas del sistema:

1. **Carga de expediente:** subida de un PDF de expediente y verificación de la extracción de asignaturas y notas.
2. **Solicitud de recomendaciones:** envío de la petición y visualización de las asignaturas recomendadas.

3. **Consulta de probabilidad de plaza:** visualización del porcentaje estimado para cada asignatura.
4. **Navegación y responsividad:** comprobación de la interfaz en escritorio y emulación de vista móvil.
5. **Manejo de errores:** subida de archivos inválidos y comprobación del tratamiento de errores.

Para cada caso se registraron el resultado funcional (éxito/fallo), observaciones sobre la experiencia y medidas de rendimiento (tiempos). Se realizaron varias iteraciones para detectar comportamientos intermitentes.

Herramientas y medición Las mediciones de latencia y tiempo de renderizado se obtuvieron con herramientas y métodos sencillos:

- Peticiones instrumentadas con `curl` / Postman para medir la latencia de la API.
- Uso de las herramientas de desarrollo del navegador (Chrome DevTools) para medir el tiempo de renderizado y carga de la vista.
- Capturas de pantalla y registros del navegador para documentar fallos o comportamientos relevantes.

Resultados Las pruebas manuales aplicadas arrojaron los siguientes resultados representativos:

- **Latencia media de la API:** aproximadamente **170 ms** por petición en el entorno de pruebas (entorno local).
- **Tiempo medio de renderizado de la vista de recomendaciones:** inferior a **900 ms** en condiciones normales de escritorio.
- **Carga y parseo del PDF:** extracciones correctas en la mayoría de los expedientes de prueba; se detectaron casos con marcos o formatos PDF no estándar que requieren pre-procesado adicional.
- **Manejo de errores:** la aplicación informa adecuadamente al usuario cuando el fichero subido no es un PDF válido o cuando faltan campos esperados.

Observaciones cualitativas Durante las pruebas manuales se observó que:

- Las recomendaciones resultan claras y la interfaz guía al usuario en el flujo principal (subir expediente → recibir recomendaciones).
- En dispositivos móviles algunos elementos de las estadísticas requieren adaptación para mejorar la legibilidad; se sugiere ajustar la visualización y probar con breakpoints adicionales.
- La experiencia ante archivos PDF no uniformes mejora si se añade una previsualización y un feedback donde se expliquen los problemas de parsing.

Limitaciones de la evaluación Se reconoce que estas pruebas, al ser realizadas por el autor y en un entorno controlado, no sustituyen a un estudio de usabilidad con usuarios reales que permita obtener conclusiones generalizables. En consecuencia, los resultados deben interpretarse como una evaluación funcional y de rendimiento inicial, útil para detectar fallos graves y optimizar la experiencia básica.

Siguientes pasos recomendados Para completar la evaluación de usabilidad en fases futuras se recomienda:

- Realizar un estudio con usuarios voluntarios representativos (estudiantes de diferentes cursos), aplicando cuestionarios estandarizados (p. ej. SUS) y recogiendo métricas de interacción.
- Implementar un sistema de feedback dentro de la aplicación para recoger opiniones reales en producción y facilitar reentrenamiento del modelo.
- Automatizar pruebas E2E (por ejemplo, con Cypress o Playwright) para detectar regresiones en la experiencia de usuario.

6.3. Rendimiento del sistema

Se han realizado pruebas en Postman y desde la aplicación web para comprobar el tiempo de respuesta de la API a un flujo de peticiones. En la mayoría de peticiones, el tiempo de respuesta es inferior a los 500 ms. Las excepciones son la probabilidad de entrada, debido a la cantidad de peticiones a la base de datos que realiza, y el subir el expediente académico, por el tiempo de procesar el pdf y limpiar los datos previo a su almacenamiento.

El tiempo de renderizado completo de la vista de recomendaciones en Angular se mantuvo por debajo de 900 ms en el 95 % de los casos, siendo la única petición con un tiempo de

carga mayor la de probabilidad de entrada como se ha mencionado anteriormente. Estos datos confirman que el sistema es apto para su uso en un entorno real con múltiples usuarios concurrentes.

Conclusiones y Líneas Futuras

7.1. Conclusiones

En este Trabajo de Fin de Grado se ha presentado el diseño e implementación de una aplicación web para apoyar la elección de asignaturas optativas, combinando un frontend en Angular con un servicio de Machine Learning en Python/FastAPI. Los principales logros alcanzados son:

- Se construyó un **pipeline de datos** que extrae, limpia y normaliza la información académica a partir de un PDF de expediente, garantizando la consistencia de los datos de entrada.
- Se compararon varios algoritmos de clasificación (k -NN, GMM).
- La API REST resultante responde en un tiempo medio de 170 ms, lo que asegura una experiencia de usuario fluida.
- La solución integró de manera satisfactoria los componentes de **recomendación** y **predicción de probabilidad de plaza**, aportando valor tanto al estudiante como al centro académico.

En conjunto, estos resultados demuestran la viabilidad de aplicar técnicas de aprendizaje supervisado y metodologías ágiles (SCRUM) en el contexto universitario, mejorando la toma de decisiones del alumnado y facilitando el análisis de demanda de asignaturas por parte de la institución.

7.2. Líneas futuras

Aunque la primera versión cumple con los objetivos planteados, se identifican varias mejoras y extensiones que podrían incorporarse en trabajos posteriores:

- **Integración de contenidos de asignaturas:** aplicar técnicas de procesamiento de lenguaje natural (NLP) para analizar temarios y combinar recomendaciones basadas en perfil con afinidad semántica.
- **Incorporación de expedientes de planes académicos recientes:** con los nuevos planes de estudio de la Escuela Técnica Superior de Ingeniería, algunas asignaturas cambian de nombre, o directamente aparecen nuevas de las cuales no se tiene soporte. Añadir registros de dichos cursos es de gran importancia para mantener la viabilidad de la aplicación.
- **Adaptación de nombres de asignaturas:** siguiendo con el punto anterior, algunas asignaturas del plan nuevo son iguales o muy similares, pero con distinto nombre, lo que hace que el modelo de aprendizaje las interprete como asignaturas distintas. Añadir un grado de similitud o equivalencia entre asignaturas podría llevar a predicciones más exactas.
- **Algoritmos avanzados de ML:** explorar métodos de deep learning (redes neuronales) o sistemas de filtrado híbrido para incrementar la precisión en entornos con grandes volúmenes de datos.
- **Feedback continuo:** implementar un sistema de retroalimentación mediante encuestas integradas en la aplicación para reajustar los modelos de recomendación con datos reales de satisfacción.

Estas líneas de trabajo permitirían consolidar y ampliar la plataforma, abordando nuevas dimensiones del problema y adaptándose a las necesidades futuras de la comunidad universitaria.

Referencias

- [1] Universidad de Málaga. “Grado en Ingeniería Informática — Universidad de Málaga.” Consultado el 5 de noviembre de 2025. (2025), dirección: <https://www.uma.es/grado-en-ingenieria-informatica/>.
- [2] M. Maphosa, W. Doorsamy y B. Paul, “A Review of Recommender Systems for Choosing Elective Courses,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 11, n.º 9, págs. 287-295, 2020. DOI: [10.14569/IJACSA.2020.0110933](https://doi.org/10.14569/IJACSA.2020.0110933). dirección: https://www.researchgate.net/publication/346077361_A_Review_of_Recommender_Systems_for_Choosing_Elective_Courses.
- [3] A. Esteban y et al, “Helping university students to choose elective courses by combining collaborative and content-based criteria,” *Expert Systems with Applications*, 2020, Ejemplo de sistema híbrido aplicado a elección de optativas.
- [4] F. Ricci, L. Rokach y B. Shapira, eds., *Recommender Systems Handbook*. Boston, MA: Springer, 2015, Capítulo y recursos de referencia sobre técnicas de recomendación.
- [5] B. Sarwar, G. Karypis, J. Konstan y J. Riedl, “An Algorithmic Framework for Performing Collaborative Filtering,” GroupLens Research, inf. téc., 2002, Framework y evaluación empírica de variantes de filtrado colaborativo. dirección: <https://files.grouplens.org/papers/algs.pdf>.
- [6] J. MacQueen, “Some Methods for Classification and Analysis of Multivariate Observations,” en *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, Descripción original del algoritmo K-means., 1967, págs. 281-297.
- [7] F. E. Öztürk y N. Demirel, “Comparison of the methods to determine optimal number of clusters,” *Veri Bilimi*, vol. 6, n.º 1, págs. 34-45, 2023.
- [8] Angular Team. “Angular — One framework. Mobile and desktop.” Consultado el 26 de agosto de 2025. (2025), dirección: <https://angular.dev/>.
- [9] Tiangolo, S. “FastAPI — High performance, easy to learn, fast to code, ready for production.” Consultado el 26 de agosto de 2025. (2024), dirección: <https://fastapi.tiangolo.com/>.
- [10] F. Pedregosa, G. Varoquaux, A. Gramfort et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, págs. 2825-2830, 2011. dirección: <https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>.
- [11] T. Cover y P. Hart, “Nearest Neighbor Pattern Classification,” *IEEE Transactions on Information Theory*, vol. 13, n.º 1, págs. 21-27, 1967. DOI: [10.1109/TIT.1967.1053964](https://doi.org/10.1109/TIT.1967.1053964).

- [12] D. W. Hosmer, S. Lemeshow y R. X. Sturdivant, *Applied Logistic Regression*, 3rd. Wiley, 2013, Referencia de referencia sobre regresión logística y evaluación de modelos.
- [13] Oracle Corporation. “Oracle SQL Developer Documentation.” Consultado el 26 de agosto de 2025. (2025), dirección: <https://docs.oracle.com/>.
- [14] OpenAI. “ChatGPT (GPT-5 Thinking mini) [Large language model].” Respuesta asistida por IA; consultado el 26 de agosto de 2025. (2025), dirección: <https://chat.openai.com/>.
- [15] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *Journal of Computational and Applied Mathematics*, vol. 20, págs. 53-65, 1987. DOI: [10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7).
- [16] D. L. Davies y D. W. Bouldin, “A cluster separation measure,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, n.º 2, págs. 224-227, 1979. DOI: [10.1109/TPAMI.1979.4766909](https://doi.org/10.1109/TPAMI.1979.4766909).
- [17] T. Calinski y J. Harabasz, “A dendrite method for cluster analysis,” *Communications in Statistics – Theory and Methods*, vol. 3, n.º 1, págs. 1-27, 1974. DOI: [10.1080/03610927408827101](https://doi.org/10.1080/03610927408827101).
- [18] I. F. Ashari, R. Banjarnahor, D. R. Farida, S. P. Aisyah, A. P. Dewi y N. Humaya, “Application of Data Mining with the K-Means Clustering Method and Davies Bouldin Index for Grouping IMDB Movies,” *Journal of Applied Informatics and Computing*, vol. 6, n.º 1, págs. 07-15, 2022, Open Access.
- [19] X. Wang e Y. Xu, “An improved index for clustering validation based on Silhouette index and Calinski–Harabasz index,” *IOP Conference Series: Materials Science and Engineering*, vol. 569, n.º 5, págs. 052 024, 2019. DOI: [10.1088/1757-899X/569/5/052024](https://doi.org/10.1088/1757-899X/569/5/052024).

Apéndice A

Manual de Instalación

A.1. Requisitos del Sistema

- Sistema Operativo: Windows o Linux.
- Git: Para clonar el repositorio.

Si va a realizar la instalación en su máquina local:

- Python: Versión 3.10+ (recomendada: 3.11.2)
- Node.js 18+ y npm.

Si va a usar Docker para instalar la aplicación:

- Docker Desktop: versión 4.7+ (recomendado)
- Docker Compose: incluido con Docker Desktop.
- No es necesario instalar manualmente las aplicaciones mencionadas en la sección de local, ya que las imágenes Docker incluyen todas las dependencias necesarias.

A.2. Guía de Instalación con Docker

A.2.1. Clonar el repositorio

```
git clone https://github.com/Jechig0/TFG.git
cd TFG
```

A.2.2. Instalar Docker Desktop y Docker Compose

Descargue Docker Desktop desde la <https://www.docker.com/products/docker-desktop>, seleccionando para su equipo la versión correspondiente.

Tras la instalación, verifique que tiene tanto docker como docker compose:

```
docker --version
docker compose version
```

Docker Desktop incluye Docker Compose por lo general, pero si por algún motivo Docker Compose no le aparece instalado, siga las instrucciones del sitio web oficial

<https://docs.docker.com/compose/install>.

A.2.3. Construir los contenedores y levantar

Con Docker Desktop ejecutandose, compile todas las imágenes y levántelas:

```
docker compose build --no-cache
docker compose up -d
```

La flag `--no-cache` no es obligatoria, sirve para reconstruir los contenedores de cero, sin usar nada que esté previamente almacenado en la memoria. Esto es útil por si realiza modificaciones en el código del proyecto, para asegurarse que al reconstruir las imágenes se implementen los cambios.

Una vez los contenedores estén contruidos y ejecutandose, puede acceder al frontend desde <http://localhost:4200>, y a la documentación en Swagger del backend desde <http://localhost:8000>

IMPORTANTE: Las imágenes Docker pueden acabar siendo pesadas y ocupar mucho espacio en memoria, si fuera necesario puede configurar desde los ajustes de Docker Desktop, sección Resources, donde se almacenan dichas imágenes.

A.2.4. Bajar los contenedores (sin borrar volúmenes)

Para dejar de ejecutar la aplicación sin borrar el contenido de Docker:

```
docker compose down
```

A.2.5. Limpiar al completo (elimina volúmenes e imágenes locales)

Para eliminar por completo el contenido de Docker para liberar capacidad:

```
docker compose down --volumes --rmi all
```

A.3. Guía de Instalación en equipo local

A.3.1. Clonar el repositorio

```
git clone https://github.com/Jechig0/TFG.git
cd TFG
```

A.3.2. Instalar Python y pip

Windows:

- Descargar la versión de Python 3.11.2
- Añadir Python al PATH durante la instalación.

Linux: Abrir una terminal y ejecutar (con la versión de Python descargada):

```
sudo apt update
sudo apt install python3 python3-pip
```

A.3.3. Configuración del entorno Python

Opcional: si no quieres modificar las librerías de Python del equipo, se recomienda crear un entorno virtual.

```
python -m venv venv
```

Activar el entorno virtual (**opcional**):

Windows:

```
venv\Scripts\activate
```

Linux:

```
source venv/bin/activate
```

A.3.4. Instalar dependencias del Backend e iniciar

Instalar las librerías requeridas de Python. En caso de error, revisar la versión de pip y Python, y volver a ejecutar el comando, o ejecutar el comando 'pip install' para cada una de las librerías.

```
pip install -r requirements.txt
```

Ejecutar el servidor del backend. Desde la carpeta raíz del repositorio:

```
cd backend
uvicorn app.main:app --reload
```

A.3.5. Instalar dependencias del Frontend y desplegar

Navegar a la carpeta de frontend. Desde la carpeta raíz del repositorio:

```
cd frontend
```

Instalar Node.js y npm. Instalar las dependencias del frontend:

```
npm install
```

Desplegar el frontend en producción:

```
ng serve
```

A.3.6. Acceder a la aplicación

La aplicación estará disponible en:

<http://localhost:4200>

Apéndice B

Manual de Usuario

B.1. Para cualquier usuario

Al iniciar la aplicación y acceder a ella, se abre la landing page. Desde ella, dependiendo del rol del usuario, tendrá que acceder a distintas partes de la web. Se puede acceder a la página de preguntas frecuentes desde la barra de navegación.

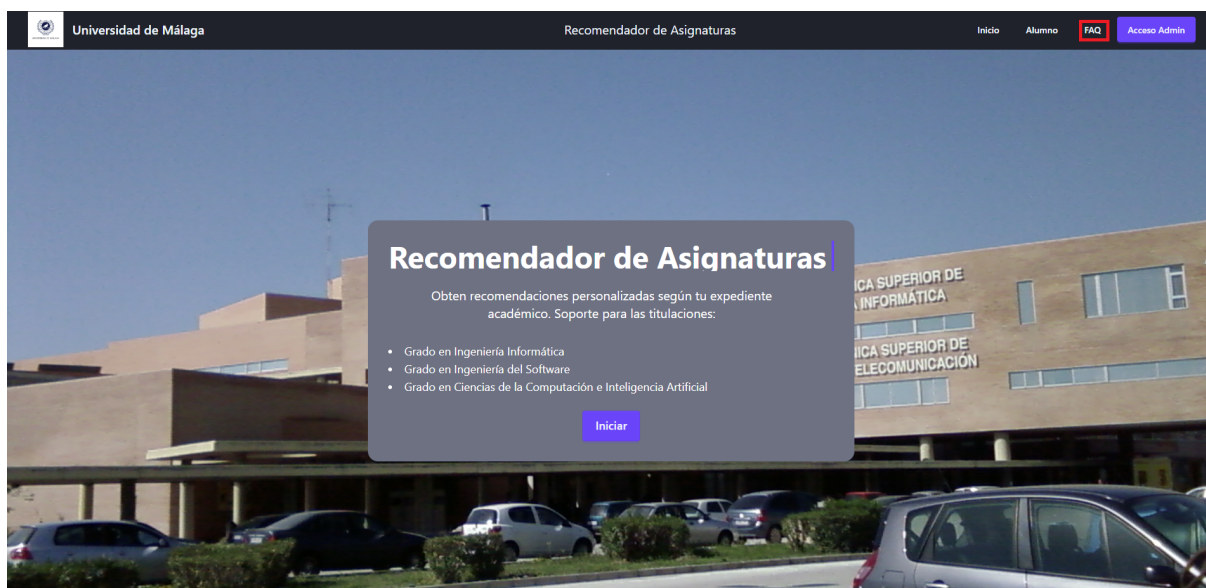


Figura 29: Acceso a preguntas frecuentes.

Desde cualquier página, se puede volver a la Landing page pulsando cualquiera de los dos siguientes botones de la barra de navegación.

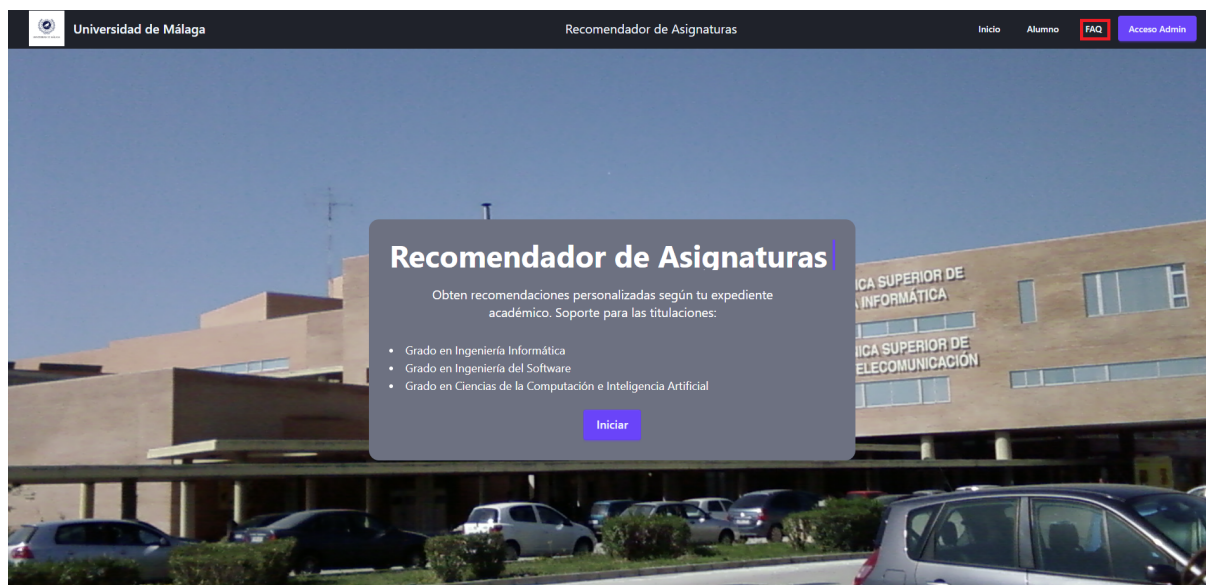


Figura 30: Vuelta a la Landing page.

B.2. Como alumno

Para iniciar sesión como alumno, puede pulsar el botón de Iniciar, o ir a la sección de Alumno de la barra de navegación.

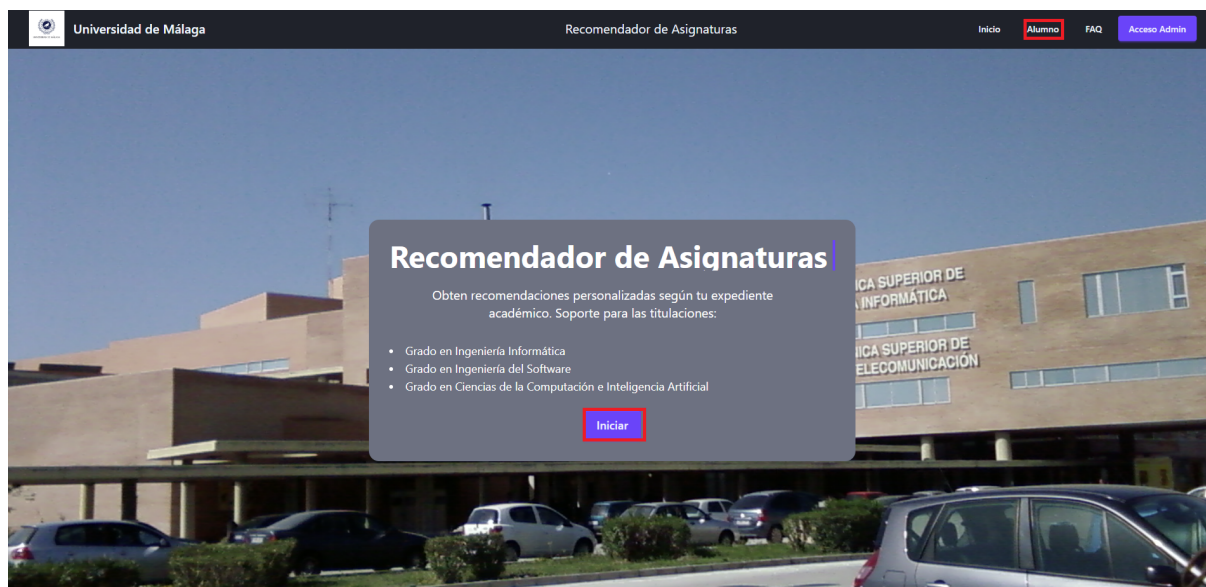
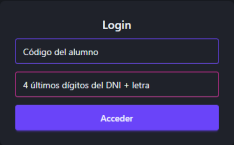


Figura 31: Acceso al inicio del usuario.

Esto redirecciona al inicio de sesión como alumno. Aquí se debe indicar el número de

expediente del alumno y los 4 últimos dígitos del DNI con la letra.



The image shows a dark-themed login interface. In the center, there is a light gray box titled "Login". Inside this box, there are two input fields: the first is labeled "Código del alumno" and the second is labeled "4 últimos dígitos del DNI + letra". Below these fields is a blue button labeled "Acceder". At the bottom of the dark background, there is a small line of text: "Toda la información recopilada en esta aplicación es tratada y almacenada de forma anónima y segura. No se almacenan datos personales del alumno."

Figura 32: Acceso al inicio del usuario.

Tras esto hay varias opciones. Primero, si los datos son correctos

- y el alumno tiene los datos guardados en la base de datos, redirecciona a su página de usuario.
- y el no tiene los datos guardados en la base de datos, redirecciona a la página de subir el expediente académico.

Si los datos son incorrectos

- el número de expediente o el DNI es incorrecto y el expediente de ese número no está en la base de datos, redirigirá a la página de subir expediente, pero si se realiza la subida, al leerlo se comprobará que los datos no coinciden y mostrará un mensaje de error.
- el número de expediente o el DNI es incorrecto y el expediente de ese número está en la base de datos, mostrará un mensaje de error ya que el DNI no coincide con el almacenado correspondiente.

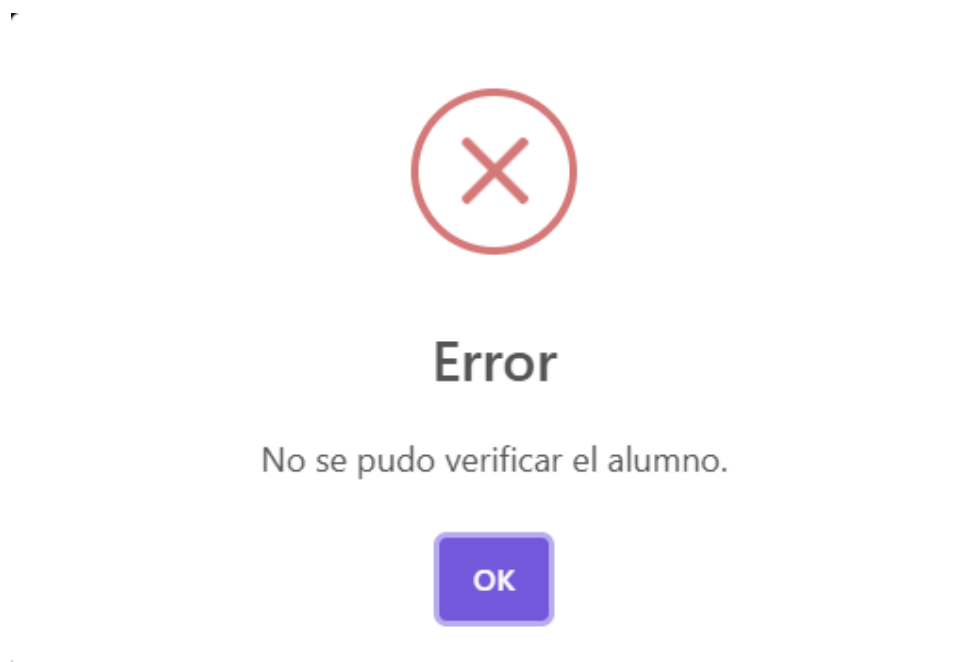


Figura 33: Mensaje de error al añadir las credenciales en el login de alumno.

Tras añadir los datos correctamente y realizar la subida del expediente en caso de ser necesaria, el alumno tiene acceso a su página, que muestra todas sus calificaciones que aparecen en el expediente y distintas optativas para su titulación sobre las que poder realizar consultas. Al pulsar en cualquiera de las asignaturas, se realizan las peticiones a la API, la cual devuelve la probabilidad de acceso y afinidad calculadas para ese alumno en esa asignatura. También aparecen en cada una de las tarjetas que muestran esta información un botón que enlaza a las preguntas frecuentes, para los alumnos que no hayan visto la opción en la barra de navegación y quieran ver cómo se calculan los datos.



Figura 34: Ejemplo de petición con lista de asignaturas y barra de búsqueda

Finalmente, también tiene el alumno a su disposición 4 botones:

- Uno para ocultar y mostrar las notas, en caso de que quiera quitar o ver los datos del expediente.
- Uno para actualizar el expediente, en caso de querer subir uno con modificaciones sin borrarlo por completo.
- Uno para borrar el expediente de la base de datos.
- Un botón de Cerrar sesión.



Figura 35: Botones del perfil del alumno.

B.3. Como administrador

El administrador debe pulsar el botón Acceso Admin, y poner su nombre de usuario y contraseña en la página de Admin Login a la cual se le redirige. Si el sistema tiene guardada esa

pareja usuario-contraseña como un administrador, se le redirigirá al panel de administrador, desde el cual podrá:

- Consultar los datos sobre las distintas consultas disponibles en el panel. Por ahora son, para todas las consultas realizadas: asignatura más consultada, asignatura con afinidad media más alta, asignatura con probabilidad de acceso media más alta, titulación que más consulta.
- Volver a ejecutar el proceso de clustering, en caso de modificación de los datos sobre los que se realiza éste, para así no tener que reiniciar necesariamente la aplicación.
- Borrar todos los datos almacenados en la tabla de alumnos y en la tabla de consultas, para así poder hacer un reiniciado de los datos almacenados cuando se considere necesario.

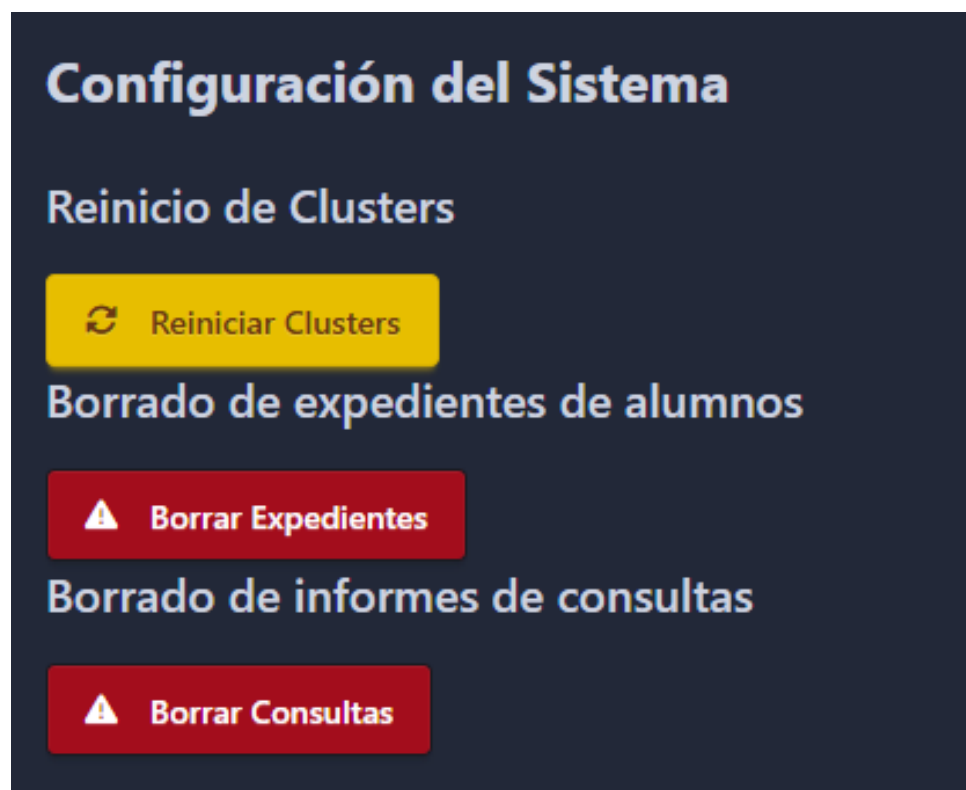


Figura 36: Botones del panel de administrador para reiniciar clústeres y vaciar las tablas de la base de datos

Valor de los años en la probabilidad de entrada

2019-20		Actual: 0.2
2020-21		Actual: 0.2
2021-22		Actual: 0.2
2022-23		Actual: 0.4

Actualizar

Figura 37: Campos del panel de administrador para ajustar el peso de los cursos académicos que se consideran a la hora de calcular la probabilidad de entrada.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga